

Monte Carlo simulation

7.1 Background

In previous chapters (e.g. Chapters 3 and 4) we have seen that option prices can be represented as expected discounted payoffs under some “risk neutral” probability measure. Monte Carlo simulation provides a method to calculate such expected values numerically in the sense that the expectation of a random variable can be approximated by the mean of a sample from its distribution.¹ Furthermore, Monte Carlo simulation can be useful to generate scenarios to evaluate the risk of a particular trading position.

Moving away from the purely probabilistic interpretation, Monte Carlo simulation can also be seen as simply a method of numerical integration. Suppose one wishes to calculate

$$I = \int_a^b f(x)dx \quad (7.1)$$

Given a method to generate random variates u_i , uniformly distributed on the interval $[a, b]$, this integral can be approximated by

$$\hat{I}_N = (b - a) \frac{1}{N} \sum_{i=1}^N f(u_i) \quad (7.2)$$

by virtue of the fact that

$$E[f(u)] = \frac{1}{b - a} \int_a^b f(x)dx \quad (7.3)$$

for a random variable U uniformly distributed on $[a, b]$.

The Monte Carlo estimator (7.2) for I is subject to a random error. This error can be quantified in a probabilistic manner using the *Central Limit Theorem*, which states that the sum of N independent, identically distributed random variables with finite mean μ and finite variance σ^2 is, for large N , approximately normally distributed with mean $N\mu$ and variance $N\sigma^2$.² Consequently, $\hat{I}_N$ will be approximately normally distributed with mean $(b - a)E[f(U)] = I$. The variance of the Monte Carlo estimator is typically not known in closed

¹ This follows from the Laws of Large Numbers, see e.g. Ash and Doléans-Dade (2000), Sections 4.11.4 and 6.2.5, or any other good textbook on probability theory.

² See e.g. Ash and Doléans-Dade (2000), Chapter 7. The Central Limit Theorem also holds for a sum of a large number of independent random vectors — such a sum will have approximately a multivariate normal distribution.

form, but can be estimated from the Monte Carlo sample via

$$\hat{\sigma}_N^2 = \frac{(b-a)^2}{N(N-1)} \sum_{i=1}^N (f(u_i) - \hat{I}_N)^2 \quad (7.4)$$

For a given confidence level $1 - \alpha$ (e.g. $1 - \alpha = 95\%$), a confidence interval for I can then be constructed around the Monte Carlo estimate $\hat{I}_N$ by

$$[\hat{I}_N - \mathcal{N}^{-1}(1 - \alpha)\hat{\sigma}_N, \hat{I}_N + \mathcal{N}^{-1}(1 - \alpha)\hat{\sigma}_N] \quad (7.5)$$

where $\mathcal{N}^{-1}(1 - \alpha)$ is the $(1 - \alpha)$ -quantile of the standard normal distribution, i.e. the true value I will lie within (7.5) with a probability of $1 - \alpha$.³ $\hat{\sigma}_N$ is also called the *standard error* of the Monte Carlo estimator. This should always be quoted in conjunction with the Monte Carlo estimate, as a Monte Carlo estimate without error bounds is not very informative.

If, as we are assuming, $f(U)$ has finite variance σ^2 ,

$$\frac{(b-a)^2}{N-1} \sum_{i=1}^N (f(u_i) - \hat{I}_N)^2 \quad (7.6)$$

will converge to $(b-a)^2\sigma^2$ and consequently $\hat{\sigma}_N$ will converge to zero with rate $\mathcal{O}(N^{-\frac{1}{2}})$ as N is increased. The width of the confidence interval (7.5) will converge to zero at the same rate, and it is in this sense that the Monte Carlo estimator can be said to converge to the true value with $\mathcal{O}(N^{-\frac{1}{2}})$. These results generalise to the multidimensional case. It is usually convenient to normalise $[a, b]$ to $[0, 1]$, so in the multidimensional case we have an integration on the unit hypercube and (7.1) becomes

$$I = \int_{[0,1]^d} f(x)dx, \quad f : [0, 1]^d \rightarrow \mathbb{R} \quad (7.7)$$

and the Monte Carlo estimator (7.2) becomes

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^N f(u_i) \quad (7.8)$$

where the u_i are d -dimensional vectors, i.e. the d components of each u_i are independent random variables uniformly distributed on the unit interval.⁴ Deterministic one-dimensional numerical integration schemes typically converge with $\mathcal{O}(N^{-2})$ and are thus clearly superior to Monte Carlo simulation for the evaluation of one-dimensional integrals. However, Monte Carlo

³ Strictly speaking, when approximating the variance of the Monte Carlo estimator using (7.4), one should be using a t -distribution with $N - 1$ degrees of freedom instead of the standard normal to determine (7.5). However, in Monte Carlo simulation N is typically sufficiently large (i.e. > 30) for this distinction to be irrelevant.

⁴ In the typical option pricing application, f would thus include mapping the u_i to normally distributed random variables representing increments of a diffusion process, mapping the diffusion process to realisations of the underlying financial variables, and mapping the financial variables to a discounted payoff of the option.

1. Initialise the variables RunningSum = 0
 RunningSumSquared = 0
 $i = 0$
2. Generate the value of the stochastic process X at each date T_k relevant to evaluate the payoff.
3. Based on the values generated in step 2, calculate the payoff.
4. Add the payoff to RunningSum and the square of the payoff to RunningSumSquared.
5. Increment the counter i .
6. If i is less than the maximum number of iterations, go to step 2.
7. Calculate the simulated mean by dividing RunningSum by the total number of iterations.
8. Calculate the variance of the simulations by dividing RunningSumSquared by the total number of iterations and subtracting the square of the mean.

Algorithm 7.1: Generic algorithm for Monte Carlo simulation

simulation does not suffer from the “curse of dimensionality” that befalls these schemes. Classical numerical integration in d dimensions converges with $\mathcal{O}(N^{-\frac{2}{d}})$, whereas the Monte Carlo estimator converges with $\mathcal{O}(N^{-\frac{1}{2}})$ regardless the dimension. In fact, for the high-dimensional multivariate normal integrals encountered in the explicit evaluation of some exotic options in Chapter 4, Monte Carlo simulation is arguably the most efficient method. This, plus the fact that it is relatively straightforward to implement, makes Monte Carlo simulation the method of choice of pricing many derivative financial instruments.

7.2 The generic Monte Carlo algorithm

In most quantitative finance applications, one is interested in asset prices which follow a stochastic process, say $X(t)$. Monte Carlo simulation of such stochastic dynamics is relatively straightforward in the case where one can directly sample from the conditional distribution of $X(t)$ given $X(s)$, $t > s$. When the asset $X_i(t)$ follows the Black/Scholes-type dynamics (4.50), the logarithmic increment (or *return*) from time s to time t is normally distributed with

$$E \left[\ln \frac{X_i(t)}{X_i(s)} \middle| \mathcal{F}_s \right] = \ln \frac{X_i(s)}{B(s, t)} - (t - s)d_{X_i} - \frac{1}{2} \int_s^t \sigma_{X_i}^2(u) du \quad (7.9)$$

$$\text{Var} \left[\ln \frac{X_i(t)}{X_i(s)} \middle| \mathcal{F}_s \right] = \int_s^t \sigma_{X_i}^2(u) du \quad (7.10)$$

```

template <class T>
class MCGatherer {
private:
    T    sum;
    T    sum2;
    size_t c;
public:
    inline MCGatherer() : sum(0.0),sum2(0.0),c(0) { };
    /// Constructor for types T which require a size argument
    /// on construction.
    inline MCGatherer(size_t array_size)
        : sum(array_size),sum2(array_size) { reset(); };
    inline void reset() { sum = 0.0; sum2 = 0.0; c = 0; };
    inline void operator+=(T add)
        { sum += add; sum2 += add*add; c++; };
    inline void operator*=(T f) { sum *= f; sum2 *= f*f; };
    inline T mean() const { return T(sum/double(c)); };
    inline T stddev() const
        { return T(sqrt((sum2/double(c)
            -mean()*mean())/double(c-1)))); };
    inline double max_stddev() const; // Must be specialised!
    inline size_t number_of_simulations() const { return c; };
};

```

Listing 7.1: Class for “gathering” Monte Carlo simulation results

and letting ξ_i denote a random draw from the standard normal distribution, one can simulate $X_i(t)$ by

$$X_i(t) = \frac{X_i(s)}{B(s,t)} \cdot \exp \left\{ (t-s)d_{X_i} - \frac{1}{2} \int_s^t \sigma_{X_i}^2(u) du + \xi_i \sqrt{\int_s^t \sigma_{X_i}^2(u) du} \right\} \quad (7.11)$$

Note that in the single asset case $X_i(t)$ can be represented in terms of a one-dimensional random variable ξ_i , regardless of the dimension of the driving Brownian motion in (4.50). When simultaneously simulating multiple assets, in general one would need to work out the variance/covariance matrix of the joint, multivariate normal distribution of the returns, and then draw from this distribution. This is necessary even if the driving Brownian motion is one-dimensional: In (4.33), even if $\sigma_X(\cdot)$ and $\sigma_Y(\cdot)$ are scalar-valued functions, the resulting correlation over any time interval $[t, T]$ is generally not one, unless $\sigma_X(\cdot)$ and $\sigma_Y(\cdot)$ are constant on $[t, T]$. By the same token, the variance/covariance matrix of the joint distribution of returns typically will have full rank (rather than rank d).

For most applications, it is sufficient to assume that the vector-valued

```

/** Template for generic Monte Carlo simulation.

    The template parameter class random_number_generator_type
    must implement a member function random(), which returns a
    realisation of the random variable of the type given by
    the template parameter class argtype.
*/
template <class argtype,class rettype,
          class random_number_generator_type>
class MCGeneric {
private:
    NormalDistribution                                N;
    /// Functor mapping random variable to Monte Carlo payoff.
    boost::function<rettype (argtype)>                f;
    random_number_generator_type                      random_number_generator;
public:
    inline MCGeneric(boost::function<rettype (argtype)> func,
                     random_number_generator_type& rng)
        : f(func),random_number_generator(rng) { };
    void simulate(MCGatherer<rettype>& mcgatherer,
                 unsigned long number_of_simulations);
    double simulate(MCGatherer<rettype>& mcgatherer,
                   unsigned long initial_number_of_simulations,
                   double required_accuracy,
                   double confidence_level = 0.95);
};

```

Listing 7.2: Template class declaration for generic Monte Carlo simulation

volatility functions $\sigma_{X_i}(\cdot)$ are piecewise constant. In this case one can simulate the driving Brownian motion component-wise, i.e. for each asset X_i we have

$$X_i(t) = \frac{X_i(s)}{B(s,t)} \cdot \exp \left\{ (t-s)d_{X_i} - \frac{1}{2}\sigma_{X_i}^2(s)(t-s) + \sum_{j=1}^d \xi^{(j)} \sigma_{X_i}^{(j)}(s) \sqrt{t-s} \right\} \quad (7.12)$$

as long as $\sigma_{X_i}(u) \equiv \sigma_{X_i}(s)$ for $u \in [s, t]$. $\xi^{(j)}$ denotes the j -th component of a d -dimensional vector ξ of independent standard normal random variates.

Algorithm 7.1 gives the general procedure for obtaining a Monte Carlo estimate. We implement this in the class templates `MCGatherer` (Listing 7.1) and `MCGeneric` (Listings 7.2 and 7.3).

The `MCGatherer` object serves to collect the Monte Carlo simulation results, i.e. steps 4 and 5 of the algorithm are implemented by the overloaded `operator+=()` and the member functions `mean()` and `stddev()` report back

```

template <class argtype,class rettype,
          class random_number_generator_type>
void MCGeneric<argtype,rettype,random_number_generator_type>::
simulate(MCGatherer<rettype>& mcgatherer,
         unsigned long number_of_simulations)
{
    unsigned long i;
    for (i=0;i<number_of_simulations;i++)
        mcgatherer += f(random_number_generator.random());
}

```

Listing 7.3: The generic Monte Carlo simulation loop

the Monte Carlo estimate and its estimated standard deviation.⁵ The template argument of `MCGatherer` determines the type of the variable representing the payoff. For a single payoff this would typically be `double`, but the template implementation is sufficiently general that this also could be `Array<double,1>`, for example, representing multiple payoffs evaluated in the same Monte Carlo simulation.

The generic Monte Carlo algorithm itself is implemented by the template `MCGeneric`, in the member function

```
void simulate(MCGatherer<rettype>& mcgatherer,
              unsigned long number_of_simulations);
```

 (7.13)

`MCGeneric` has three template arguments. The first is `class argtype`, which is the type of random variable needed to evaluate the payoff — typically, this would be `double` in the univariate case or `Array<double,1>` in the multivariate case. The second is `class rettype`, which is the type of the variable representing the payoff. The third is `class random_number_generator_type`, which must be a class with a member function

```
argtype random()
```

 (7.14)

which returns a draw of the random variable of the required type — in the univariate, normally distributed case, one could use

```
ranlib::Normal<double>
```

 (7.15)

from the Blitz++ library.

Each `MCGeneric` object is initialised at construction with a reference to a random number generator of the required type and a function f mapping the random variable in each draw of the simulation to the (discounted) payoff, where f is represented by the Boost Library functor

```
boost::function<double (argtype)> f;
```

 (7.16)

In the main simulation routine in Listing 7.3, Algorithm 7.1 is now very compact:

⁵ Encapsulating part of the algorithm in the `MCGatherer` class follows the *strategy* design pattern. See Joshi (2004) for a detailed discussion.

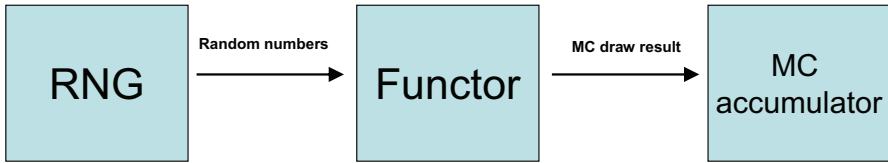


Figure 7.1 *Interaction of objects in the generic Monte Carlo algorithm*

- The initialisations in Step 1 were done by the constructor of the `MCGatherer` object.
- The function call `random_number_generator.random()` generates the random variable in Step 2,
- to which the functor `f` is applied to calculate the payoff in Step 3. Note that `f` encapsulates the entire mapping from random variables to discounted payoff, thus the generic implementation of the Monte Carlo algorithm abstracts from the *stochastic model* (which maps basic random variables to the dynamics of the underlying assets) and any particular *financial instrument* (which maps the prices of underlying assets to a discounted payoff).
- Step 4 is embodied in the overloaded `operator+=()` of the `MCGatherer` object.
- Steps 5 and 6 are managed by the `for` loop.
- Steps 7 and 8 are completed on demand by the member functions `mean()` and `stddev()` of `MCGatherer`.

Thus Steps 2, 3 and 4 are implemented by the statement

```
mcgatherer += f(random_number_generator.random());
```

 (7.17)

in `MCGeneric::simulate()`, i.e. through the interaction of the random number generator, the functor `f` and the Monte Carlo result accumulator `mcgatherer`, as illustrated in Figure 7.1.

Encapsulating the collection of the simulation results in an `MCGatherer` object also makes it easy to implement a Monte Carlo estimate to a desired accuracy (rather than a fixed number of simulations) in the member function of `MCGeneric`,

```
double simulate(MCGatherer<rettype>& mcgatherer,
               unsigned long initial_number_of_simulations,
               double required_accuracy,
               double confidence_level)
```

as Listing 7.4 demonstrates. After performing an initial set of simulations, the achieved accuracy can be calculated via

$$\mathcal{N}^{-1}(1 - \alpha)\hat{\sigma}_n$$
 (7.18)

where $1 - \alpha$ is the desired confidence level and $\hat{\sigma}_n$ the standard deviation of

```

template <class argtype,class rettype,
          class random_number_generator_type>
double MCGeneric<argtype,rettype,random_number_generator_type>::
simulate(MCGatherer<rettype>& mcgatherer,
         unsigned long initial_number_of_simulations,
         double required_accuracy,
         double confidence_level)
{
    unsigned long n = initial_number_of_simulations;
    simulate(mcgatherer,initial_number_of_simulations);
    double d = N.inverse(confidence_level);
    double current_accuracy = d * mcgatherer.max_stddev();
    while (required_accuracy<current_accuracy) {
        double q = current_accuracy/required_accuracy;
        q *= q;
        unsigned long additional_simulations = n * q - n;
        if (!additional_simulations) break;
        simulate(mcgatherer,additional_simulations);
        n += additional_simulations;
        current_accuracy = d * mcgatherer.max_stddev(); }
    return current_accuracy;
}

```

Listing 7.4: Generating a Monte Carlo estimate to a desired accuracy

the Monte Carlo estimator.⁶ Additional simulations are then performed until (7.18) is reduced to the desired level. The number of additional simulations required can be predicted by noting that (7.18) will decrease approximately proportionately to the square root of the number of simulations, so we set the size of the next batch of simulations to

$$\left(\frac{\text{current accuracy}}{\text{required accuracy}} \right)^2 n - n \quad (7.19)$$

As a simple example of using `MCGeneric` and `MCGatherer`, consider the Black/Scholes model implemented in `MCGenericTest.cpp`,⁷ where the stock price under the risk neutral measure at time T is given by

$$S(T) = S(0) \exp \left\{ \left(r - \frac{1}{2} \sigma^2 \right) T + \sigma W(T) \right\} \quad (7.20)$$

where $W(T)$ is normally distributed with mean 0 and variance T . We want to

⁶ In order to cater for multiple payoffs evaluated in the same Monte Carlo simulation (e.g. a `rettype` of `Array<double,1>`), $\hat{\sigma}_n$ is accessed via `MCGatherer::max_stddev()`. The implementation of this template member function is specialised for different cases of `rettype`: `MCGatherer<double>::max_stddev()` simply returns `MCGatherer::stddev()`, which is the standard deviation of the Monte Carlo estimator as defined in (7.4), while `MCGatherer<Array<double,1>>::max_stddev()` returns the maximum standard deviation of the multiple Monte Carlo estimators collected by this `MCGatherer` object.

⁷ Available on the website for this book.

Paths	MC value	95% CI lower bound	95% CI upper bound	Difference in standard errors
100	14.1060	9.8154	18.3966	-1.4738
400	15.8688	13.5168	18.2209	-1.4558
1600	16.6760	15.5070	17.8450	-1.7933
6400	17.9812	17.3623	18.6002	0.0817
25600	18.0816	17.7796	18.3835	0.7139
102400	18.0850	17.9346	18.2354	1.4710
409600	17.9905	17.9156	18.0653	0.8784
1638400	17.9628	17.9255	18.0002	0.5435

Table 7.1 *Convergence of the Monte Carlo estimator for the price of a European call option to the closed form solution. The closed form price is 17.9505.*

use Monte Carlo simulation to calculate the price of a European call option via the expected discounted value

$$E \left[e^{-rT} \max(0, S(T) - K) \right] \quad (7.21)$$

We can write the discounted payoff inside the expectation as a function f of a standard normal random variable ξ :

$$f(\xi) = e^{-rT} \max \left(0, S(0) \exp \left\{ \left(r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} \xi \right\} - K \right) \quad (7.22)$$

This function can be bound to a `boost::function<double (double)>` functor and passed to an `MCGeneric<double,ranlib::NormalUnit<double> >` object, which is then used to run the Monte Carlo simulations.

Table 7.1 gives the output of `MCGenericTest.cpp` for 100 up to 1638400 simulations.⁸ One clearly sees the convergence of the Monte Carlo estimate to the closed form Black/Scholes price. As per (7.5), the 95% confidence interval around the Monte Carlo estimate tightens proportionally to the square root of the number of simulations. The last column gives the difference between the Monte Carlo estimate and the closed form value in terms of the number of standard deviations $\hat{\sigma}_n$, i.e.

$$\frac{\text{MC value} - \text{closed form value}}{\hat{\sigma}_n} \quad (7.23)$$

This is a useful statistic when testing the correctness of a Monte Carlo simulator against a closed form solution. The probability that the absolute value of (7.23) is greater than 2 is about 5%. Thus an absolute value greater than 2 is “suspicious” and indicates a possible bias in the simulation. If one subsequently increases the number of simulations, this value will either drop back

⁸ The initial stock price was 100, the volatility 30%, the continuously compounded interest rate 5%. The price is for a European call option with strike 100 maturing in 1.5 years.

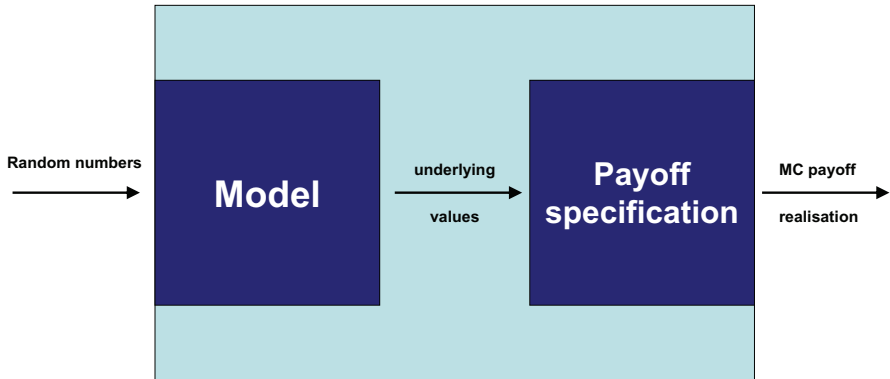


Figure 7.2 *The two-stage mapping of random numbers to asset prices, to the discounted payoff*

below 2 (as in Table 7.1) or continue to increase, reaching ever larger and more improbable values. The latter effectively proves the presence of a bias in the simulator. Whenever possible, an implementation of a Monte Carlo simulator should be tested on a payoff with a known closed form expectation, before proceeding to use the simulator to calculate expectations for which no closed form is available.

7.3 Simulating asset price processes following geometric Brownian motion

In many cases, the payoff that one wishes to evaluate may depend on the value of several assets at several points in time. It is thus convenient to view steps 2 and 3 in Algorithm 7.1 as a two-stage mapping, as illustrated in Figure 7.2: First, a set of random numbers is used to generate the required asset prices at the required points in time, based on the stochastic model. Second, the asset prices thus obtained are mapped to the discounted payoff.

We encapsulate the two steps in the template class `MCMMapping`, the public interface of which is given in Listing 7.5. In turn, the mapping of random numbers to asset prices is delegated to the template parameter class `price_process`, and the asset prices are then mapped to a payoff by a class derived from the abstract base class `MCPayoff`.

The logic of `MCMMapping` is best demonstrated in an example. Consider the option to exchange one asset for another in Theorem 4.7. There are two underlying assets, the dynamics of which are given by (4.50), i.e. by two geometric Brownian motions driven by the same (generally multidimensional) Wiener process. We can represent each asset by the `BlackScholesAsset` class introduced in Chapter 4. Assume that the dimension of the driving Wiener

```

template <class price_process, class random_variable>
class MCMapping {
public:
    /// Constructor.
    MCMapping(MCPayoff& xpayoff,
              price_process& xprocess,
              const TermStructure& xts,
              int xnumeraire_index = -1);
    /// Choose the numeraire asset for the equivalent martingale
/// measure under which the simulation is carried out.
    bool set_numeraire_index(int xnumeraire_index);
    /// The function mapping a realisation of the (often
/// multidimensional) random variable x to the discounted
/// payoff.
    double mapping(random_variable x);
    /// The function mapping a realisation of the (often
/// multidimensional) random variable x to multiple discounted
/// payoffs.
    Array<double,1> mappingArray(random_variable x);
};

```

Listing 7.5: The public interface of the template class MCMapping

process is two⁹ and for simplicity set volatilities to be constant using the class `ConstVol`.¹⁰ Thus we could initialise the two assets by

```

Array<double,1> sgm1(2);
sgm1 = 0.3 0.0;
ConstVol vol1(sgm1);
BlackScholesAsset asset1(&vol1,100.0);
Array<double,1> sgm2(2);
sgm2 = 0.2 0.1;
ConstVol vol2(sgm2);
BlackScholesAsset asset2(&vol2,100.0);

```

(7.24)

To represent the joint stochastic process of assets with Black/Scholes-type dynamics and to map random variables representing increments of geometric Brownian motion to a realisation of the asset price paths, we use the class `GeometricBrownianMotion`. This class has a constructor, which takes as its argument a set of underlying assets, represented by a `std::vector<const BlackScholesAsset*>`; thus we need the following sequence of statements:¹¹

⁹ This is by no means a necessary assumption. However, a dimension of one would lead to instantaneously perfectly correlated asset dynamics, while a dimension greater than two is unnecessary when there are only two assets.

¹⁰ In practical applications in the presence of a term structure of volatility one might instead use `PiecewiseConstVol`.

¹¹ The implementation of `GeometricBrownianMotion` is discussed below.

```

class MCPayoff {
public:
    Array<double,1> timeline; ///Timeline collecting all event dates.
    Array<int,2> index; ///A 2 x N matrix of indices, where each
                        ///column represents the indices of an
                        ///(asset,time) combination affecting the payoff.
    inline MCPayoff(const Array<double,1>& xtimeline,
                    const Array<int,2>& xindex)
        : timeline(xtimeline),index(xindex) { };
    inline MCPayoff(const Array<double,1>& xtimeline,int number_of_indices)
        : timeline(xtimeline),index(2,number_of_indices) { };
    inline MCPayoff(int number_of_event_dates,int number_of_indices)
        : timeline(number_of_event_dates+1),index(2,number_of_indices) { };
    ///Calculate discounted payoff.
    virtual double operator()(const Array<double,1>& underlying_values,
                             const Array<double,1>& numeraire_values) = 0;
    ///Allow for multidimensional payoff (i.e. portfolio) with meaningful
    ///default (one-dimensional) behaviour.
    virtual Array<double,1>
        payoffArray(const Array<double,1>& underlying_values,
                    const Array<double,1>& numeraire_values);
};

```

Listing 7.6: The public interface of the class MCPayoff

```

std::vector<const BlackScholesAsset*> underlying;
underlying.push_back(&asset1);
underlying.push_back(&asset2);
GeometricBrownianMotion gbm(underlying);

```

(7.25)

In order to map the random variables representing increments of Brownian motion to the Monte Carlo payoff(s), we then construct an instance of `MCMapping`. In addition to an object representing the price process (e.g. `GeometricBrownianMotion`), the constructor of `MCMapping` requires a reference to an instance of a class derived from `MCPayoff` (Listing 7.6). `MCPayoff` has a member `operator()()`, which calculates the discounted payoff based on a set of values of the underlying assets and the numeraire asset at a given set of “event dates.” In the barrier option example below, the implementation of a class derived from `MCPayoff` is discussed in detail — in the present context, let it be sufficient to note that the `MBinaryPayoff` class introduced in Chapter 4 has `MCPayoff` as a parent class. Thus payoffs of the type (4.51) can be specified in a manner allowing Monte Carlo and closed-form Black/Scholes pricing on the same payoff object, which is helpful when using the Black/Scholes model as a control variate (see Section 7.6.2, below.) The easiest way to obtain an `MCPayoff` for an option to exchange one asset for another is to first construct an instance of `exotics::Margrabe`, which returns the corresponding `MCPayoff` via the member function `get_payoff()`, e.g.:

```
double maturity = 2.5;
double strike = 1.1;
FlatTermStructure ts(0.06,0.0,maturity);
exotics::Margrabe margrabe(asset1,asset2,
                           0.0,maturity,1.0,strike,ts);
boost::shared_ptr<MCPayoff> margrabe_payoff =
    margrabe.get_payoff();
```

(7.26)

This allows us to construct an instance of `MCMapping` by

```
MCMapping<GeometricBrownianMotion,Array<double,2> >
mc_mapping(*margrabe_payoff,gbm,ts,numeraire_index);
```

(7.27)

where the optional argument `numeraire_index` gives the index of the asset, which is to serve as the numeraire, in the set of assets in the `GeometricBrownianMotion` object. The default is a numeraire index of -1 , which corresponds to taking the riskfree asset as the numeraire (i.e. deterministic discounting) — see Chapter 8 for a discussion of Monte Carlo simulation under a different choice of numeraire.¹²

The second template argument of `MCMapping` is in this case `Array<double,2>`: In order to generate a simulated path of the assets, (dimension of the driving Brownian motion) $\times$ (number of time steps) independent normal random variables are required. `MCMapping` then provides the function mapping the random variables to the Monte Carlo payoff, e.g.

```
boost::function<double(Array<double,2>>>
func = boost::bind(&MCMapping<GeometricBrownianMotion,
                        Array<double,2> >::mapping,
                        &mc_mapping,1);
```

(7.28)

To encapsulate the required array of normal random variables, we create the template class `RandomArray`, e.g. if the dimension of the driving Brownian motion is 2 and there are N time steps,

```
ranlib::NormalUnit<double> normalRNG;
RandomArray<ranlib::NormalUnit<double>,double>
random_container(normalRNG,2,N);
```

(7.29)

i.e. `random_container.random()` will return a $2 \times N$ `Array` of independent standard normal random variables. Then we are in a position to construct the `MCGeneric` object with

```
MCGeneric<Array<double,2>,
double,
RandomArray<ranlib::NormalUnit<double>,double>
> mc(func,random_container);
```

(7.30)

which is used to generate the Monte Carlo estimate for the option price in exactly the same manner as in the single-asset, single-time horizon case in Section 7.2. Thus the core Monte Carlo algorithm remains the same not

¹² It is worth noting that one can conduct a Monte Carlo simulation under the “real-world” or “physical” measure in a manner that is consistent with simulation and pricing under a risk neutral measure by selecting as the numeraire an asset, the volatility of which is equal to the market price of risk. For discussion of the numeraire associated with the physical measure, see Bajeux-Besnainou and Portait (1997).

```

class down_and_out_call : public MCPayoff {
public:
    double strike, barrier;
    down_and_out_call(const Array<double,1>& T,
                      int underlying_index,
                      double xstrike, double xbarrier);
    /// Calculate discounted payoff.
    virtual double
        operator()(const Array<double,1>& underlying_values,
                   const Array<double,1>& numeraire_values);
};

down_and_out_call::
down_and_out_call(const Array<double,1>& T,
                  int underlying_index,
                  double xstrike, double xbarrier)
: MCPayoff(T, T.extent(firstDim)-1,
           strike(xstrike), barrier(xbarrier))
{
    firstIndex idx;
    secondIndex jdx;
    index = idx * (jdx+1);
}

double down_and_out_call::
operator()(const Array<double,1>& underlying_values,
          const Array<double,1>& numeraire_values)
{
    int i;
    bool indicator = true;
    double result = 0.0;
    for (i=0; i<underlying_values.extent(firstDim); i++)
        if (underlying_values(i) <= barrier) indicator = false;
    if (indicator) result = numeraire_values(0) /
        numeraire_values(numeraire_values.extent(firstDim)-1) *
        std::max(0.0, underlying_values(
            underlying_values.extent(firstDim)-1)-strike);
    return result;
}

```

Listing 7.7: An MCPayoff for a down-and-out call option.

only conceptually, but also in implementation.¹³ All that has changed is the random variables being fed into the simulation (a `RandomArray` vs. straight `ranlib::NormalUnit<double>` in the previous example) and the functor mapping the random variables to the discounted payoff.

¹³ The code for this example is given in the file `MCMargrabe.cpp` on the website.

Similarly, we can price a down-and-out barrier option by feeding the appropriate class derived from `MCPayoff` into `MCMapping`.¹⁴ Again, one could obtain the required `MCPayoff` by first constructing an instance of `exotics::DiscreteBarrierOut` with

```
exotics::DiscreteBarrierOut
    down_and_out_call(asset1,maturity,strike,
                      barrier,ts,1,-1);
```

(7.31)

and then calling the member function `get_payoff()`. As before, the `MCPayoff` then allows us to construct an `MCMapping`, which provides the functor to feed into `MCGeneric`.

However, for didactic purposes let us create from scratch a class derived from `MCPayoff`, as given in Listing 7.7. Let N be the number of (not necessarily equidistant) time steps at which the barrier is monitored between now (time 0) and maturity. For simplicity, we choose the time steps to be equidistant in the present example. We create an instance of this class `down_and_out_call_payoff` by

```
Array<double,1> T(N+1);
firstIndex idx;
double dt = maturity/N;
T = idx*dt;
down_and_out_call_payoff DOpayoff(T,0,strike,barrier);
```

(7.32)

The constructor of `down_and_out_call_payoff` takes as its first argument the timeline T of “event dates,” which is an `Array<double,1>` of increasing time points starting at 0 (now) and ending at the option maturity. The barrier is monitored at each event date and the payoff occurs at maturity. The second argument is the index of the underlying asset (to allow for a `price_process` in `MCMapping` representing more than one underlying asset), followed by the option strike and down-and-out barrier. The constructor initialises the data members `strike` and `barrier`, and passes the timeline through to the constructor of the parent class `MCPayoff`. It also initialises the data member `index` of `MCPayoff`, which represents the (asset, time) index combinations which need to be simulated, i.e. in this case asset 0 at each time T_i , $0 < i \leq N$, so

$$\text{index} = \begin{pmatrix} 0 & 0 & \cdots & 0 \\ 1 & 2 & \cdots & N \end{pmatrix}$$

The member `operator()()` is defined to calculate the discounted payoff from the simulated values of the underlying asset and the numeraire at each date, which `MCMapping` requests from `price_process` and passes to an instance of this class `down_and_out_call` derived from `MCPayoff`.

The instance `DOpayoff` of the class `down_and_out_call` is passed to the constructor of `MCMapping`, which creates the functor for use in `MCGeneric`, and the Monte Carlo simulation proceeds as before.

When one wishes to generate a Monte Carlo estimate for the price of an instrument incorporating several `MCPayoffs`, or value a portfolio (either in total

¹⁴ The code for this example is given in the file `MCDownandout.cpp` on the website.

```

class GeometricBrownianMotion {
public:
    GeometricBrownianMotion(std::vector<const BlackScholesAsset*>&
        xunderlying);
    ~GeometricBrownianMotion();
    /// Query the dimension of the process.
    int dimension() const;
    /// Query the number of factors driving the process.
    inline int factors() const { return dW.extent(firstDim); };
    /// Set process timeline.
    bool set_timeline(const Array<double,1>& timeline);
    /// Get process timeline.
    inline const Array<double,1>& get_timeline() const
        { return *timeline_; };
    inline int number_of_steps() const
        { return T->extent(firstDim)-1; };
    /// Generate a realisation of the process under the martingale
    /// measure associated with deterministic bond prices.
    void operator()(Array<double,2>& underlying_values,
        const Array<double,2>& x,
        const TermStructure& ts);
    /// Generate a realisation of the process under the martingale
    /// measure associated with a given numeraire asset.
    void operator()(Array<double,2>& underlying_values,
        Array<double,1>& numeraire_values,
        const Array<double,2>& x,
        const TermStructure& ts,
        int numeraire_index);
};

```

Listing 7.8: The public interface of `GeometricBrownianMotion`.

or component-wise) of several `MCPayoffs` in the same Monte Carlo simulation, one can use class `MCPayoffList`. In addition to the constructor, the virtual member `operator()()` and function `payoffArray()` declared by the parent class `MCPayoff`, `MCPayoffList` provides the member function

```

void push_back(boost::shared_ptr<MCPayoff> xpayoff,
    double xcoeff = 1.0);

```

(7.33)

to add an `MCPayoff` to the list. It performs various housekeeping operations to ensure that the data members `timeline` and `index` of the parent class `MCPayoff` are updated so that `MCMapping` requests all the necessary (asset, time) pairs from the `price_process`, and that the correct `Arrays` of underlying asset and numeraire values are passed to the member `operator()()` and function `payoffArray()` of the `MCPayoffs` in the list. The member `operator()()` and function `payoffArray()` of `MCPayoffList` call the member `operator()()` of each `MCPayoff` in the list, the difference being that `payoffArray()` returns an `Array<double>` containing the result of `operator()()` for each `MCPayoff`,


```

template <class price_process, class random_variable>
double MCMapping<price_process, random_variable>::
mapping(random_variable x)
{
    int i;
    /* Using the random draw x, generate the values of the price
       process at the dates in the required timeline,
       under the martingale measure associated with the chosen
       numeraire. */
    if (numeraire_index >= 0)
        process(underlying_values, numeraire_values,
                x, ts, numeraire_index);
    else process(underlying_values, x, ts);
    // Map underlying values to the payoff.
    for (i=0; i<mapped_underlying_values.extent(firstDim); i++)
        mapped_underlying_values(i) =
            underlying_values(payoff.index(0,i), payoff.index(1,i));
    // Calculate the discounted payoff.
    return payoff(mapped_underlying_values, numeraire_values);
}

```

Listing 7.9: The member function `mapping()` of `MCMapping`.

while `operator()()` of `MCPayoffList` returns the sum of the individual results.¹⁵ Note that `MCPayoff` relates to `MCPayoffList` via both inheritance and composition, i.e. `MCPayoffList` is an `MCPayoff` and contains `MCPayoffs`.

In keeping with the view of each iteration of Monte Carlo simulation as a mapping of a draw of random variables to a discounted option payoff, the object representing the price process has the task of converting the draw of random variables into a realisation of the price path of the underlying asset(s). For geometric Brownian motion, it is straightforward to generate price paths based on the exact (i.e. lognormal) transition densities, and thus there is no bias in the simulation at any level of discretisation. This is implemented in the class `GeometricBrownianMotion`, the public interface of which is given in Listing 7.8. After constructing an instance of `GeometricBrownianMotion` from a vector of `BlackScholesAssets`, the timeline of dates, for which realisations of the underlying assets need to be observed, can be set using the member function `set_timeline()`. When `GeometricBrownianMotion` is used in conjunction with `MCMapping`, `MCMapping` passes the timeline of required observation dates from `MCPayoff` into `GeometricBrownianMotion::set_timeline()`. The member function `mapping()` of `MCMapping` (see Listing 7.9) maps the draw of random variables to a discounted payoff; in order to generate the required price path of the underlying asset(s), `mapping()` calls the overloaded `operator()` of the price process class, e.g. `GeometricBrownianMotion`. De-

¹⁵ See `MCCallPut.cpp` on the website for example usage.

pending on whether the simulation is being carried out under the usual risk neutral probability measure under deterministic interest rates, or under a martingale measure associated with taking one of the underlying assets as the numeraire, `mapping()` calls (under the usual measure)

```
void operator()(Array<double,2>& underlying_values,
               const Array<double,2>& x,
               const TermStructure& ts);
```

 (7.34)

or (for one of the underlying assets as the numeraire)

```
void operator()(Array<double,2>& underlying_values,
               Array<double,1>& numeraire_values,
               const Array<double,2>& x,
               const TermStructure& ts,
               int numeraire_index);
```

 (7.35)

where `numeraire_index` refers to the index of the numeraire asset in the vector of underlying assets from which the instance of `GeometricBrownianMotion` was constructed. Thus one can change from geometric Brownian motion to alternative stochastic dynamics for the underlying asset(s) by replacing `GeometricBrownianMotion` by a different price process class, which must appropriately implement the member function `set_timeline()` and the two versions (7.34) and (7.35) of the `operator()`. This is done in Chapter 8.

7.4 Discretising stochastic differential equations

If it is not possible (or computationally too expensive) to sample directly from the distribution of the underlying assets, one may instead resort to discretising and simulating the stochastic differential equations (SDE) describing the asset dynamics.

Assume that asset dynamics are given by a vector-valued Itô process

$$dX(t) = a(X(t), t)dt + b(X(t), t)dW(t) \quad (7.36)$$

i.e. $X(t)$ is an N -dimensional vector of asset prices, $W(t)$ is a d -dimensional standard Brownian motion, $a(\cdot, \cdot)$ is a vector-valued function (dimension N), and $b(\cdot, \cdot)$ is a matrix-valued function (dimension $N \times d$). The drift function a and volatility function b are assumed to satisfy sufficient conditions for (7.36) to have a unique strong solution.¹⁶

The simplest way to discretise and simulate (7.36) is the *Euler scheme*. Consider a discretisation of the timeline $0 = t_0 < t_1 < \dots < t_M$. The stochastic dynamics of X can be approximated by

$$X(t_{m+1}) = X(t_m) + a(X(t_m), t_m)(t_{m+1} - t_m) \quad (7.37)$$

$$+ b(X(t_m), t_m)\sqrt{t_{m+1} - t_m}\xi_m \quad (7.38)$$

where each ξ_m is a d -dimensional vector of independent standard normal variates.

¹⁶ See e.g. Karatzas and Shreve (1991), Section 5.2.

The discretisation (7.37) is only approximate and will typically show *discretisation bias*. That is, a Monte Carlo estimator based on $X(t)$ simulated via (7.37) will converge to a value different from the desired expectation $E[f(X(t))]$ as the number of simulations is increased. The size of this bias depends on the chosen discretisation of the timeline. As $t_{m+1} - t_m$ goes to zero, the discretised dynamics (7.37) will converge to the true dynamics (7.36). There are two types of convergence to be considered, *strong* and *weak*.¹⁷

Definition 7.1 1. Denote by $X(T)$ a stochastic process and by $\hat{X}_\delta(T_M)$, $T_M = T$, its discretised approximation with maximum step size

$$\delta = \max_m t_{m+1} - t_m$$

Then $\hat{X}_\delta(T_M)$ is said to converge strongly to $X(T)$ at time T if

$$\lim_{\delta \rightarrow 0} E[|X(T) - \hat{X}_\delta(T_M)|] = 0$$

Furthermore, $\hat{X}_\delta$ is said to converge strongly with order $\gamma > 0$ if

$$E[|X(T) - \hat{X}_\delta(T_M)|] \leq C\delta^\gamma \quad \forall \delta < \delta_0$$

for some constants C , δ_0 .

2. Denote by $\mathcal{C}$ a class of test functions $g : \mathbb{R}^N \rightarrow \mathbb{R}$. $\hat{X}_\delta(T_M)$ is said to converge weakly to $X(T)$ at time T if

$$\lim_{\delta \rightarrow 0} |E[g(X(T))] - E[g(\hat{X}_\delta(T_M))]| = 0 \quad \forall g \in \mathcal{C}$$

Furthermore, $\hat{X}_\delta$ is said to converge weakly with order $\beta > 0$ if

$$|E[g(X(T))] - E[g(\hat{X}_\delta(T_M))]| \leq C\delta^\beta \quad \forall g \in \mathcal{C}, \delta < \delta_0$$

for some constants C , δ_0 .

Kloeden and Platen (1992) show that the Euler scheme usually (i.e. under some mild additional technical conditions) converges with weak order $\beta = 1$ and with strong order $\gamma = 0.5$. Thus, when choosing to implement a particular discretisation scheme, it is important to consider whether one requires a good pathwise (strong) approximation or whether a good (weak) convergence of expectations of some function is what is required. Clearly, the latter is the priority when pricing derivative financial instruments, while the former would come into play simulating hedging effectiveness, for example.

Strong order $\gamma = 1$ convergence can be achieved by adding further (second order) terms to (7.37). This is the *Milstein scheme*. In the one-dimensional

¹⁷ For a detailed discussion of these convergence concepts, see Kloeden and Platen (1992).

case it is given by

$$\begin{aligned}
 X(T_{m+1}) &= X(t_m) + a(X(t_m), t_m)(t_{m+1} - t_m) \\
 &\quad + b(X(t_m), t_m)\sqrt{t_{m+1} - t_m}\xi_m \\
 &\quad + \frac{1}{2}b(X(t_m), t_m)\frac{\partial b(X(t_m), t_m)}{\partial X(t_m)}(t_{m+1} - t_m)(\xi_m^2 - 1)
 \end{aligned} \tag{7.39}$$

and thus does not require any additional draws of random variates beyond those required by (7.37).

The choice of scheme (Euler, Milstein, or higher order) and refinement of the time line discretisation to reduce discretisation bias can be encapsulated in a class representing a price process, i.e. a class providing the necessary functionality of a valid `price_process` template parameter to the class template `MCMapping`.

7.5 Predictor–Corrector methods

As pricing derivative financial instruments is a much more common requirement than simulating hedging effectiveness, we focus on methods to further improve the order of weak convergence.¹⁸ Predictor–corrector methods can be implemented with relative ease to achieve this. As above, the reader is referred to Kloeden and Platen (1992) for rigorous proofs of the results reproduced here.

Discretisation bias in the Euler scheme stems from the fact that the drift coefficient $a(X(t), t)$ and the diffusion coefficient $b(X(t), t)$ are assumed to be constant on the discretisation time step $[t_m, t_{m+1}]$ and equal to their values at the beginning of the time step, i.e. $a(X(t_m), t_m)$ and $b(X(t_m), t_m)$, thus not taking into account the variation of a and b on $[t_m, t_{m+1}]$ due to the variation of $X(t)$. Predictor–corrector methods aim to achieve a better approximation of the true dynamics of $X(t)$ by first generating a *predictor* $\bar{X}(t_{m+1})$ of $X(t_{m+1})$ (e.g. using an Euler scheme increment) and then *correcting* the initial drift and diffusion coefficients based on $X(t_m)$ at the beginning of the time step by terms based on the predicted value $\bar{X}(t_{m+1})$ at the end of the time step.

A predictor–corrector method with weak convergence of order 1 for (7.36) is given by

$$\begin{aligned}
 &X(t_{m+1}) \\
 &= X(t_m) + (\alpha \bar{a}_\eta(\bar{X}(t_{m+1}), t_{m+1}) + (1 - \alpha) \bar{a}_\eta(X(t_m), t_m))(t_{m+1} - t_m) \\
 &\quad + \sum_{j=1}^d (\eta b_j(\bar{X}(t_{m+1}), t_{m+1}) + (1 - \eta) b_j(X(t_m), t_m)) \Delta W_j
 \end{aligned} \tag{7.40}$$

Recall that X is an N -dimensional vector, and so are $\bar{a}_\eta(\cdot, \cdot)$ and $b_j(\cdot, \cdot)$. (7.40)

¹⁸ Note that in order to *calculate* hedge ratios (as opposed to simulating hedging effectiveness), weak convergence is again the relevant concept, as a hedge ratio is determined by the sensitivity of the price to an underlying risk factor.

represents a family of schemes parameterised in $\alpha, \eta \in [0, 1]$; a typical choice would be $\alpha = 0.5$ and $\eta = 0$. The adjusted drift coefficient $\bar{a}_\eta$ is given by¹⁹

$$\bar{a}_\eta = a - \eta \sum_{j=1}^d \sum_{k=1}^N b_{k,j} \frac{\partial b_j}{\partial x_k} \quad (7.41)$$

and the predictor $\bar{X}(t_{m+1})$ is given by

$$\bar{X}(t_{m+1}) = X(t_m) + a(X(t_m), t_m)(t_{m+1} - t_m) + \sum_{j=1}^d b_j(X(t_m), t_m) \Delta W_j \quad (7.42)$$

The ΔW_j represent the Brownian motion increments on the time step $[t_m, t_{m+1}]$, and thus are Gaussian random variables with mean zero and variance $\Delta = t_{m+1} - t_m$. However, it is also admissible (and for small time steps possibly more efficient) to choose the ΔW_j as two-point distributed random variables with

$$P(\Delta W_j = \sqrt{\Delta}) = P(\Delta W_j = -\sqrt{\Delta}) = \frac{1}{2} \quad (7.43)$$

In the one-dimensional case (i.e. $d = N = 1$) with $\alpha = 0.5$ and $\eta = 0$, (7.40) becomes

$$\begin{aligned} X(t_{m+1}) &= X(t_m) + \frac{1}{2} (a(\bar{X}(t_{m+1}), t_{m+1}) + a(X(t_m), t_m)) (t_{m+1} - t_m) \\ &\quad + b(X(t_m), t_m) \Delta W \end{aligned}$$

with predictor

$$\bar{X}(t_{m+1}) = X(t_m) + a(X(t_m), t_m)(t_{m+1} - t_m) + b(X(t_m), t_m) \Delta W \quad (7.44)$$

Depending on the structure of a particular Itô process (7.36), the order of weak convergence achieved by (7.40) can be greater than the guaranteed order of 1. A predictor-corrector method with a guaranteed order 2 weak convergence can also be constructed. In this case, the corrector is given by

$$X(t_{m+1}) = X(t_m) + \frac{1}{2} (a(\bar{X}(t_{m+1}), t_{m+1}) + a(X(t_m), t_m)) (t_{m+1} - t_m) + \Psi_m$$

where

$$\begin{aligned} \Psi_m &= \sum_{j=1}^d \left(b_j(X(t_m), t_m) + \frac{1}{2} L^0 b_j(X(t_m), t_m) (t_{m+1} - t_m) \right) \Delta W_j \\ &\quad + \frac{1}{2} \sum_{j_1=1}^d \sum_{j_2=1}^d L^{j_1} b_{j_2}(X(t_m), t_m) (\Delta W_{j_1} \Delta W_{j_2} + V_{j_1, j_2}) \end{aligned}$$

¹⁹ This is based on what Kloeden and Platen (1992) call the “fully implicit” Euler scheme. See p. 496ff in this reference.

with the operators L^0 and L^j defined by

$$L^0 = a \frac{\partial}{\partial x} + \frac{1}{2} b^2 \frac{\partial^2}{\partial x^2} \quad L^j = \sum_{k=1}^N b_{k,j} \frac{\partial}{\partial x_k}$$

The predictor is given by

$$\begin{aligned} \bar{X}(t_{m+1}) &= X(t_m) + a(X(t_m), t_m)(t_{m+1} - t_m) + \Psi_m \\ &\quad + \frac{1}{2} L^0 a(X(t_m), t_m)(t_{m+1} - t_m)^2 \\ &\quad + \frac{1}{2} \sum_{j=1}^d L^j a(X(t_m), t_m) \Delta W_j (t_{m+1} - t_m) \end{aligned}$$

The ΔW_j can be chosen as independent Gaussian random variables with mean zero and variance $\Delta = t_{m+1} - t_m$, or alternatively as independent three-point distributed random variables with

$$P(\Delta W = \sqrt{3\Delta}) = P(\Delta W = -\sqrt{3\Delta}) = \frac{1}{6}, \quad P(\Delta W = 0) = \frac{2}{3}$$

The V_{j_1, j_2} are independent two-point distributed random variables satisfying

$$P(V_{j_1, j_2} = \Delta) = P(V_{j_1, j_2} = -\Delta) = \frac{1}{2}$$

for $j_2 = 1, \dots, j_1 - 1$ and

$$V_{j_1, j_1} = -\Delta, \quad V_{j_1, j_2} = -V_{j_2, j_1}$$

for $j_2 = j_1 + 1, \dots, d$ and $j_1 = 1, \dots, d$.

As with the choice of discretisation scheme in the previous section, the implementation of predictor–corrector methods can be encapsulated in a class representing a price process.

7.6 Variance reduction techniques

We have seen that the accuracy of a Monte Carlo estimate — as measured by the width of, say, a 95% confidence interval around the estimate — improves in proportion to the square root of the number of simulations. Variance reduction methods aim to reduce $\hat{\sigma}_N$ (the standard deviation of the Monte Carlo estimator given by (7.4)) by reducing the standard deviation σ of $f(U)$ — halving σ will improve the accuracy of the Monte Carlo estimate by as much as would a fourfold increase in the number of simulations.

7.6.1 Antithetic sampling

The variance reduction method which is arguably the easiest to implement is based on antithetic variates. For symmetric distributions, antithetic values are generated by mirroring on the mean, i.e. if u is a random variate uniformly

```

template <class argtype,class rettype,
          class random_number_generator_type>
void MCGeneric<argtype,rettype,random_number_generator_type>::
simulate(MCGatherer<rettype>& mcgatherer,
         unsigned long number_of_simulations)
{
    unsigned long i;
    if (!antithetic)
        for (i=0;i<number_of_simulations;i++)
            mcgatherer += f(random_number_generator.random());
    else {
        for (i=0;i<number_of_simulations;i++) {
            argtype rnd = random_number_generator.random();
            rettype res = f(rnd);
            res = 0.5 * (res + f(antithetic(rnd)));
            mcgatherer += res; }
    }
}

```

Listing 7.10: The generic Monte Carlo simulation loop with antithetic sampling

```

/// Antithetic variables mirrored around zero.
template <class T>
T normal_antithetic(T arg)
{
    return T(-arg);
}

/// Antithetic for uniform [0,1].
template <class T>
T uniform_antithetic(T arg)
{
    return T(1.0 - arg);
}

```

Listing 7.11: Templates for the typical antithetic functors

distributed on $[a, b]$, its antithetic variate is given by

$$u' = a + b - u$$

In the typical case that u is uniformly distributed on $[0, 1]$, this reduces to $u' = 1 - u$. Where standard normal random variates z are required, one can set $z = N^{-1}(u)$ and $z' = N^{-1}(1 - u)$, or equivalently (and more efficiently) $z' = -z$. Given a random variate and its antithetic value, each draw in the Monte Carlo sample is calculated by

$$\bar{f}_{u_i} = \frac{1}{2}(f(u_i) + f(u'_i)) \quad (7.45)$$

```
// ----- create underlying asset -----
Array<double,1> sgm1(1);
sgm1 = sgm;
ConstVol vol(sgm1);
BlackScholesAsset stock(&vol,S,0.03);
std::vector<const BlackScholesAsset*> underlying;
underlying.push_back(&stock);
// ----- closed form option price -----
cout << "S: " << S << "\nK: " << K << "\nr: " << r
      << "\nT: " << mat << "\nvolatility: " << sgm << endl;
double CFcall = stock.option(mat,K,r);
cout << "Closed form call: " << CFcall << endl;
double CFprice = CFcall;
// flat term structure for discounting
FlatTermStructure ts(r,0.0,mat+10.0);
```

Listing 7.12: Pricing a European call option in a model driven by geometric Brownian motion (i.e. Black/Scholes) — setup and closed form price. Full code available in `AntitheticExample.cpp` on the website.

The Monte Carlo estimator $\hat{I}_N$ in (7.2) then becomes

$$\hat{I}_N = (b - a) \frac{1}{N} \sum_{i=1}^N \bar{f}(u_i)$$

and its variance

$$\hat{\sigma}_N = \frac{(b - a)^2}{N(N - 1)} \sum_{i=1}^N (\bar{f}(u_i) - \hat{I}_N)^2$$

By (7.45) this requires twice as many evaluations of the function $f(\cdot)$ as in plain Monte Carlo, so antithetic variates lead to a worthwhile variance reduction if

$$\text{Var}[\bar{f}(u)] < \frac{1}{2} \text{Var}[f(u)]$$

which is equivalent to

$$\text{Cov}[f(u), f(u')] < 0$$

If $f(\cdot)$ is monotonic in u , this condition is satisfied, but when $f(\cdot)$ is symmetric around the mean of u , antithetic variates would lead to no variance reduction at all. For most models and payoff functions encountered in finance, monotonicity of $f(\cdot)$ will hold.²⁰

In the C++ implementation, at the level of the generic Monte Carlo algorithm class template `MCGeneric`, $f(u)$ is calculated for each draw of u and the result collected in an `MCGatherer`. For antithetic sampling, $f(u)$ is replaced by

²⁰ A notable exception would be the “quadratic Gaussian” models such as the one discussed by Beaglehole and Tenney (1991), which involve a quadratic form of the driving state variable.


```

unsigned long n = minpaths;
// instantiate random number generator
ranlib::NormalUnit<double> normalRNG;
RandomArray<ranlib::NormalUnit<double>,double>
random_container2(normalRNG,1,1); // 1 factor, 1 time step
// instantiate stochastic process
GeometricBrownianMotion gbm(underlying);
// 95% quantile for confidence interval
boost::math::normal normal;
double d = boost::math::quantile(normal,0.95);
// boost functor to convert random variates to their antithetics
// (instantiated from template)
boost::function<Array<double,2> (Array<double,2>>)
    antithetic = normal_antithetic<Array<double,2> >;
// instantiate MCGatherer objects to collect simulation results
MCGatherer<double> mcgatherer;
MCGatherer<double> mcgatherer_antithetic;
// instantiate MCPayoff object
MCEuropeanCall mc_call(0,mat,0,call_strike);
// instantiate MCMMapping and bind to functor
MCMMapping<GeometricBrownianMotion,Array<double,2> >
    mc_mapping2(mc_call,gbm,ts,numeraire_index);
boost::function<double (Array<double,2>>)> func2 = boost::bind(
    &MCMMapping<GeometricBrownianMotion,Array<double,2> >::mapping,
    &mc_mapping2,_1);
// instantiate generic Monte Carlo algorithm object
MCGeneric<Array<double,2>,double,
    RandomArray<ranlib::NormalUnit<double>,double> >
    mc2(func2,random_container2);
MCGeneric<Array<double,2>,double,
    RandomArray<ranlib::NormalUnit<double>,double> >
    mc2_antithetic(func2,random_container2,antithetic);

```

Listing 7.13: Pricing a European call option by Monte Carlo simulation in a model driven by geometric Brownian motion (i.e. Black/Scholes), with and without antithetic sampling — instantiation of the required Monte Carlo classes. Full code available in `AntitheticExample.cpp` on the website.

$\bar{f}(u)$ as per (7.45), and thus `MCGeneric` is the appropriate place to implement this. We add the functor

(7.46)

```
boost::function<argtype (argtype)> antithetic;
```

as a private data member of `MCGeneric`. This functor serves to convert a realisation of the random variable of type `argtype` into its antithetic value. Listing 7.3 then becomes Listing 7.10, which implements (7.45) if `antithetic` is present.

The public interface of `MCGeneric` is amended by adding

```

cout << "European call option\nPaths,Closed form value,MC value,
      95% CI lower bound,95% CI upper bound,
      Difference in standard errors,";
cout << "Antithetic MC value,95% CI lower bound,
      95% CI upper bound,Difference in standard errors,
      CI width,Antithetic CI width\n";
// run Monte Carlo for different numbers of simulations
while (mcgatherer.number_of_simulations()<maxpaths) {
    mc2.simulate(mcgatherer,n);
    // half as many paths for antithetic
    mc2_antithetic.simulate(mcgatherer_antithetic,n/2);
    cout << mcgatherer.number_of_simulations() << ',' << CFprice
        << ',' << mcgatherer.mean() << ','
        << mcgatherer.mean()-d*mcgatherer.stddev() << ','
        << mcgatherer.mean()+d*mcgatherer.stddev() << ','
        << (mcgatherer.mean()-CFprice)/mcgatherer.stddev() << ',';
    cout << mcgatherer_antithetic.mean() << ','
        << mcgatherer_antithetic.mean()
            -d*mcgatherer_antithetic.stddev() << ','
        << mcgatherer_antithetic.mean()
            +d*mcgatherer_antithetic.stddev() << ','
        << (mcgatherer_antithetic.mean()-CFprice)/
            mcgatherer_antithetic.stddev() << ',';
    cout << 2.0*d*mcgatherer.stddev() << ','
        << 2.0*d*mcgatherer_antithetic.stddev() << ',' << endl;
    n = mcgatherer.number_of_simulations(); }

```

Listing 7.14: Pricing a European call option by Monte Carlo simulation in a model driven by geometric Brownian motion (i.e. Black/Scholes), with and without antithetic sampling — running Monte Carlo for different numbers of simulations. Full code available in `AntitheticExample.cpp` on the website.

```

void set_antithetic(boost::function<argtype (argtype)>
    antifunc) (7.47)

```

and the overloaded constructor

```

MCGeneric(boost::function<rettype (argtype)> func,
    random_number_generator_type& rng,
    boost::function<argtype (argtype)>
    antifunc) (7.48)

```

to turn on antithetic sampling (it can be turned off again by passing an empty functor to `set_antithetic()`).

Templates for the typical antithetic functors are given in `QFRandom.hpp` (see Listing 7.11). Thus if we require a functor to convert an array of standard normal random variates into their antithetic values, we can instantiate it by

```

boost::function<Array<double,2> (Array<double,2>)>
    antithetic = normal_antithetic<Array<double,2> > (7.49)

```

Listings 7.12 to 7.14 show how to price a European call option by Monte Carlo

	Without antithetic sampling			With antithetic sampling			
Paths	MC value	95% conf. interval		MC value	95% conf. interval		Ratio
500	12.8025	10.9738	14.6312	16.3369	14.6648	18.009	1.09
1000	13.1877	11.8794	14.4959	15.4681	14.3071	16.6290	1.13
2000	14.4451	13.4658	15.4245	15.1199	14.3093	15.9305	1.21
4000	14.9054	14.1933	15.6175	15.0886	14.5199	15.6573	1.25
8000	15.0729	14.5746	15.5712	15.0503	14.6517	15.4488	1.25
16000	15.1899	14.8420	15.5378	15.2321	14.9503	15.5138	1.23
32000	15.3473	15.1005	15.5940	15.4917	15.2893	15.6940	1.22
64000	15.3164	15.1429	15.4899	15.3567	15.2145	15.4989	1.22
128000	15.1874	15.0652	15.3095	15.2779	15.1774	15.3784	1.22
256000	15.2299	15.1435	15.3163	15.2825	15.2115	15.3536	1.22
512000	15.2225	15.1615	15.2836	15.2181	15.1681	15.2681	1.22
1024000	15.2235	15.1803	15.2666	15.1735	15.1383	15.2088	1.22
2048000	15.2049	15.1744	15.2354	15.1742	15.1493	15.1991	1.22
4096000	15.1979	15.1764	15.2195	15.1680	15.1504	15.1856	1.22
8192000	15.1997	15.1845	15.2150	15.1840	15.1715	15.1965	1.22

Table 7.2 Monte Carlo simulation of a European call option, with and without antithetic sampling. Antithetic values are generated using half as many paths as the “plain” values. Initial stock price 100, strike 100, time to maturity 1.5 years, interest rate 5% continuously compounded, volatility 30%. Black/Scholes price 15.1867. The last column shows the ratio between the width of the 95% confidence intervals with and without antithetic sampling.

simulation in a model driven by geometric Brownian motion (i.e. Black/Scholes), with and without antithetic sampling. Listing 7.12 instantiates the class representing the underlying asset and calculates the closed-form Black/Scholes price. Listing 7.13 assembles the classes required to perform the Monte Carlo simulation: the random number generator, the stochastic process, the payoff, the instance of `MCMapping` mapping the random number to realisations of the underlying asset price and to the option payoff, the antithetic functor, the generic Monte Carlo algorithm and instances of `MCGatherer` to hold the simulation results.

This example program was used to generate Table 7.2, which compares the standard deviations of the resulting Monte Carlo estimators. The antithetic estimates are based on half as many simulations as the “plain” estimates (since they require about twice as much computational effort per simulation). Antithetic sampling in this case improves the accuracy of the Monte Carlo estimate by about 22%.

7.6.2 Control variates

As before, given a (possibly multidimensional) random variable U , we wish to calculate $E[f(U)]$ by Monte Carlo simulation. Suppose that for another function g , $E[g(U)]$ can be evaluated analytically. If $f(U)$ and $g(U)$ are closely correlated, then $g(U)$ represents an effective control variate. Naively, if $f(U)$

```

template <class target_price_process,
          class controlvariate_price_process,
          class random_variable>
double MCControlVariateMapping<target_price_process,
                               controlvariate_price_process,
                               random_variable>
::mapping(random_variable x)
{
    return target.mapping(x) - controlvariate.mapping(x)
        + controlvariate_values_sum;
}

template <class target_price_process,
          class controlvariate_price_process,
          class random_variable>
double MCControlVariateMapping<target_price_process,
                               controlvariate_price_process,
                               random_variable>
::mappingArray(random_variable x)
{
    return target.mappingArray(x) - controlvariate.mappingArray(x)
        + controlvariate_values;
}

```

Listing 7.15: Code implementing `mapping()` (for a scalar payoff) and `mappingArray()` for the template class `MCControlVariateMapping`

and $g(U)$ are of similar variance and setting $a = 0$, $b = 1$, we can replace the Monte Carlo estimator (7.2) by

$$\hat{I}_N = \left(\frac{1}{N} \sum_{i=1}^N (f(u_i) - g(u_i)) \right) + E[g(U)] \quad (7.50)$$

and the variance of the difference of the two closely correlated random variables $f(U)$ and $g(U)$ is much lower than the variance of $f(U)$ alone:

$$\text{Var}[f(U) - g(U)] = \text{Var}[f(U)] + \text{Var}[g(U)] - 2\rho\sqrt{\text{Var}[g(U)]\text{Var}[f(U)]} \quad (7.51)$$

where ρ is the correlation between $f(U)$ and $g(U)$.

This approach to control variate variance reduction is implemented in the class template `MCControlVariateMapping`, which can replace `MCMMapping` in the implementation of “plain” Monte Carlo as it provides the public member functions `mapping()` and `mappingArray()` used to “map” a random variate draw to a discounted payoff. As noted previously, the “plain” mapping (and thus $f(\cdot)$) encapsulates the model, the discounting and the payoff. A second “plain” mapping represents $g(\cdot)$ and encapsulates the control variate model, discounting and payoff. `MCControlVariateMapping` brings the two plain mappings together, encapsulating $f(\cdot)$, $g(\cdot)$ and the control variate calculation

(7.50). Thus the control variate $g(\cdot)$ may differ from the target $f(\cdot)$ in terms of its “model” (i.e. how the input random variates are mapped to realisations of the underlying financial variables), its discounting or its payoff, the only requirement being that $E[g(U)]$ is known exactly and $f(U)$ and $g(U)$ are closely correlated.

Therefore the constructor of `MCControlVariateMapping` is

```
MCControlVariateMapping(
    MCMapping<target_price_process,
    random_variable>& xtarget,
    MCMapping<controlvariate_price_process,random_variable>&
    xcontrolvariate,
    const Array<double,1>& xcontrolvariate_values)
```

where `target_price_process` and `controlvariate_price_process` may differ, but must be driven by the same `random_variable`. The constructor argument `xcontrolvariate_values` contains the exact value of $E[g(U)]$, which may in general be a vector.

The code implementing `mapping()` (for a scalar payoff) and `mappingArray()` (for a vector-valued payoff) is given in Listing 7.15, and the full implementation of `MCControlVariateMapping` can be found in the file `MCEngine.hpp` on the website.

The textbook example for using control variates in MonteCarlo simulation in finance is the case of the arithmetic average (or Asian) option. This option does not have a closed-form pricing formula in the Black/Scholes model, as the arithmetic average of lognormal random variables is not lognormal. However, the geometric average is lognormal and closely correlated to the arithmetic average, so an option on the geometric average can be priced in closed form and provides an effective control variate for the arithmetic average option.

The code using `MCControlVariateMapping` to price a call on the arithmetic average of the underlying stock price, i.e. with payoff

$$\max \left(0, \frac{1}{n} \sum_{j=1}^n S(t_j) - K \right)$$

for an underlying stock price $S(\cdot)$ observed at a discrete set of dates t_j , is given in the file `CVTest.cpp` on the website. The closed-form solution for the geometric average option is calculated using the “Quintessential Option Pricing Formula” from Section 4.3. The template-based approach makes it easy to implement the Monte Carlo simulation in its “plain” version (Case 1), with antithetic sampling (Case 2), with control variates (Case 3), and with both antithetic sampling and control variates (Case 4). Table 7.3 compares all four cases. For a given number of simulations, antithetic sampling and control variate variance reduction each increase the computational effort by about a factor of two, so in order to make the results comparable, in each row of the table Cases 2 and 3 involve half as many simulations as Case 1, and Case 4 involves one quarter as many simulations. Use of the control variate

	Case 1: plain MC		Case 2: antithetic MC	
Paths	MC value	CI width	MC value	CI width
200	20.4445	9.8290	14.4162	5.0281
800	15.9943	3.6264	16.1170	2.7406
3200	16.4933	1.8148	16.0282	1.4640
12800	16.2209	0.8862	16.1802	0.7528
51200	16.1333	0.4409	16.0277	0.3741
204800	15.9870	0.2189	16.0116	0.1866
819200	16.0712	0.1103	16.0925	0.0939
3276800	16.0996	0.0553	16.0591	0.0467

	Case 3: MC with control variate		Case 4: antithetic MC with control var.	
Paths	MC value	CI width	MC value	CI width
200	16.6326	1.7580	15.9488	0.9979
800	16.2901	1.0580	16.0943	0.8569
3200	15.9721	0.4697	16.1298	0.4842
12800	16.0441	0.2420	16.1502	0.2278
51200	16.1128	0.1248	16.1698	0.1161
204800	16.0941	0.0605	16.1092	0.0558
819200	16.1015	0.0304	16.0819	0.0270
3276800	16.1004	0.0152	16.0899	0.0136

	Ratios between CI widths			
Paths	Case 1/2	Case 1/3	Case 1/4	Case 3/4
200	1.9548	5.5911	9.8500	1.7617
800	1.3232	3.4277	4.2320	1.2346
3200	1.2396	3.8637	3.7478	0.9700
12800	1.1773	3.6625	3.8903	1.0622
51200	1.1788	3.5320	3.7992	1.0757
204800	1.1730	3.6164	3.9217	1.0844
819200	1.1744	3.6240	4.0896	1.1285
3276800	1.1825	3.6289	4.0691	1.1213

Table 7.3 *Pricing a call option with strike 102 maturing in 2.25 years on the arithmetic average of a stock price observed at 11 equally spaced time points. The model is Black/Scholes with initial stock price 100, 63.64% volatility and an interest rate of 5%. Pricing is done by ‘plain’ Monte Carlo (Case 1), with antithetic sampling (Case 2), with control variates (Case 3), and with both antithetic sampling and control variates (Case 4). “CI width” refers to the width of a 95% confidence interval around the Monte Carlo estimate.*

improves accuracy by a factor of about 3.6 without antithetic sampling and 4 with antithetic sampling, which for a given accuracy translates to a reduction in computing time by a factor of about 13 and 16, respectively.

7.6.3 Optimal control variate weights

One notes that (7.50) can be generalised to

$$\hat{I}_N = \left(\frac{1}{N} \sum_{i=1}^N (f(u_i) - \beta g(u_i)) \right) + \beta E[g(U)] \quad (7.52)$$

An optimal choice of β minimises $\text{Var}[f(U) - \beta g(U)]$, in other words, β is the slope of a linear regression of $f(u)$ on $g(u)$, and thus given by

$$\beta = \frac{\text{Cov}[f(U), g(U)]}{\text{Var}[g(U)]} \quad (7.53)$$

Using β chosen optimally in this way to evaluate (7.52), the variance of the original Monte Carlo estimator (7.2) is reduced by a factor of

$$1 - (\text{Corr}[f(U), g(U)])^2 \quad (7.54)$$

where $\text{Corr}[f(U), g(U)]$ is the correlation between $f(U)$ and $g(U)$. In other words, in order to achieve the same accuracy as (7.52) using the “plain” estimator (7.2), $1/(1 - (\text{Corr}[f(U), g(u)])^2)$ as many simulations are required.

Typically, the covariance between $f(U)$ and $g(U)$, $\text{Cov}[f(U), g(U)]$, will not be known analytically, so we have to calculate β using sample estimates for variance and covariance, setting

$$\begin{aligned} \bar{f} &= \frac{1}{N} \sum_{i=1}^N f(u_i) \\ \bar{g} &= \frac{1}{N} \sum_{i=1}^N g(u_i) \\ \hat{\beta}_N &= \frac{\sum_{i=1}^N (f(u_i) - \bar{f})(g(u_i) - \bar{g})}{\sum_{i=1}^N (g(u_i) - \bar{g})^2} \end{aligned} \quad (7.55)$$

Using $\hat{\beta}_N$ in (7.52) introduces some bias if $\hat{I}_N$ and $\hat{\beta}_N$ are calculated on the same set of simulations. This bias is avoided if $\hat{\beta}_N$ is calculated on an independent (possibly smaller) set of simulations, separately from $\hat{I}_N$.²¹

(7.52) can be generalised further to allow for multiple control variates, i.e.

$$\hat{I}_N = \left(\frac{1}{N} \sum_{i=1}^N \left(f(u_i) - \sum_{j=1}^J \beta_j g_j(u_i) \right) \right) + \sum_{j=1}^J \beta_j E[g_j(U)] \quad (7.56)$$

²¹ However, experience appears to indicate that this bias tends to be small. See Glasserman (2004), Section 4.1.3, for a discussion of this issue.

```

double MCGatherer<Array<double,1> >::CVestimate(int target,int
CV,double CV_expectation) const
{
    double b = CVweight(target,CV);
    return mean(target) - b*(mean(CV) - CV_expectation);
}

double MCGatherer<Array<double,1> >::CVweight(int target,int CV)
const
{
    double b = 0.0;
    if (CVon) {
        b = (covar(target,CV)/number_of_simulations() -
mean(target)*mean(CV)) / (variance(CV)*number_of_simulations()); }
    return b;
}

```

Listing 7.16: The member functions `CVestimate()` and `CVweight()` for Monte Carlo estimates using one control variate

with J control variates $g_j(\cdot)$. The optimal weights β_j can be estimated by multivariate linear regression: Denote by S_g the $J \times J$ matrix with jk entry

$$\frac{1}{N-1} \left(\sum_{i=1}^N g_j(u_i) g_k(u_i) - N \bar{g}_j \bar{g}_k \right)$$

and by S_{fg} the J -dimensional vector with j -th entry

$$\frac{1}{N-1} \left(\sum_{i=1}^N g_j(u_i) f(u_i) - N \bar{g}_j \bar{f} \right)$$

Then $\hat{\beta}_N$ is the J -dimensional vector which solves linear system of equations

$$S_g \hat{\beta}_N = S_{fg} \quad (7.57)$$

One should note that the use of multiple control variates exacerbates small-sample issues (see e.g. Section 4.1.3 of Glasserman (2004)), but in modern computer implementations one would typically choose N in the thousands, while J would be a small integer (typically less than ten), so the impact of these issues can be expected to be negligible.

The appropriate place to implement control variates with optimal weights is a specialisation of the `MCGatherer` class template, i.e.²²

```
template<> class MCGatherer<Array<double,1> > (7.58)
```

is used to collect simulation results for a set of payoffs, where some payoffs may be used as control variates for one “target” payoff. Use of control variates can be turned on or off with the member function

²² See also the files `MCGatherer.hpp` and `MCGatherer.cpp` on the website.


```

double MCGatherer<Array<double,1> >::
CVestimate(int target,
           const Array<int,1>& CV,
           const Array<double,1>& CV_expectation) const
{
    int i;
    int n = CV.extent(blitz::firstDim);
    Array<double,2> b(CVweight(target,CV));
    double result = mean(target);
    for (i=0;i<n;i++)
        result -= b(i,0) * (mean(CV(i)) - CV_expectation(i));
    return result;
}

Array<double,2> MCGatherer<Array<double,1> >::
CVweight(int target,const Array<int,1>& CV) const
{
    int i,j;
    int n = CV.extent(blitz::firstDim);
    Array<double,2> covCV(n,n),cov(n,1),b(n,1);
    b = 0.0;
    if (CVon) {
        for (i=0;i<n;i++) {
            double meani = mean(CV(i));
            for (j=0;j<i;j++) {
                covCV(i,j) = covCV(j,i) =
                    covar(CV(i),CV(j))/number_of_simulations() -
                    meani*mean(CV(j)); }
            covCV(i,i) = covar(CV(i),CV(i))/number_of_simulations() -
                meani*meani;
            cov(i,0) = covar(target,CV(i))/number_of_simulations() -
                mean(CV(i))*mean(target); }
        interfaceCLAPACK::SolveLinear(covCV,b,cov); }
    return b;
}

```

Listing 7.17: The member functions CVestimate() and CVweight() for Monte Carlo estimates using several control variates

```
void set_control_variate(bool cv) (7.59)
```

If control variates have been turned on, the accumulation operator,

```
void operator+=(const Array<double,1>& add) (7.60)
```

records the information necessary to calculate S_g and S_{fg} .

Note that even with control variates simulation turned on, the member functions

```

Array<double,1> mean() const
Array<double,1> stddev() const
Array<double,1> variance() const

```

(7.61)

Paths	MC value	CI width	CV MC value	CV CI width	Acc. gain
100	0.098103	1.63E-02	0.101249	5.75E-04	28.29
400	0.100593	8.19E-03	0.101322	2.86E-04	28.62
1600	0.101214	4.30E-03	0.101166	1.81E-04	23.79
6400	0.101297	2.17E-03	0.101146	8.48E-05	25.63
25600	0.100603	1.09E-03	0.101104	4.90E-05	22.20
102400	0.101176	5.44E-04	0.101110	2.39E-05	22.77
409600	0.101147	2.71E-04	0.101109	1.18E-05	23.08
1638400	0.101118	1.36E-04	0.101103	5.91E-06	23.01

Table 7.4 *Pricing an option with payoff (7.66), strike 1.47876, power $\alpha = 0.1$ maturing in 1.5 years. Closed form value of the option is 0.101104. The model is Black/Scholes with initial stock price 100, 42.43% volatility and an interest rate of 5%. Pricing is done by “plain” Monte Carlo and with five standard call options with strikes 37.5, 50, 62.5, 75 and 125 as control variates. “CI width” refers to the width of a 95% confidence interval around the Monte Carlo estimate.*

return the vector of “plain” Monte Carlo estimates, Monte Carlo estimate standard deviations and variances, respectively. The template specialisation also provides the member functions

```
double mean(int i) const
double stddev(int i) const
double variance(int i) const
```

(7.62)

to access the mean, standard deviation and variance of the Monte Carlo estimate for a particular element in the vector of Monte Carlo estimates collected by `MCGatherer<Array<double,1> >`.

Monte Carlo estimates using one control variate are accessed using the member function

```
double CVestimate(int target,int CV,
                  double CV_expectation) const
```

(7.63)

where `target` is the index of the target payoff $f(\cdot)$ and `CV` is the index of the control variate payoff $g(\cdot)$ in the vector of Monte Carlo estimates, and `CV_expectation` is the exact value of $E[g(U)]$. `CVestimate()` calls the member function `CVweight(target,CV)` to calculate β using (7.55) and then calculates the Monte Carlo estimate using (7.52), see Listing 7.16.

The member function

```
double CVestimate_stddev(int target,int CV) const
```

(7.64)

uses (7.54) to calculate the standard deviation of the Monte Carlo estimate. The file `CVtest.cpp` on the website demonstrates this version of the control variate simulation for the case of an option on the arithmetic average, with an option on the geometric average as the control variate.

For multiple control variates, `CVestimate()` is overloaded to

```
double CVestimate(int target,
                  const Array<int,1>& CV,
                  const Array<double,1>&
                  CV_expectation) const
```

(7.65)

where **CV** now is an **Array** of indices of the control variate payoffs $g_j(\cdot)$ in the vector of Monte Carlo estimates, and **CV_expectation** is the vector of exact values $E[g_j(U)]$. This version of **CVestimate()** implements (7.56), where the optional control variate weights are calculated in an overloaded version of **CVweight(target, CV)** by solving (7.57), see Listing 7.17.

To get a reliable confidence interval indicating the accuracy of a Monte Carlo estimate with multiple control variates, it is recommended to run a number of simulations to estimate the optional control variate weights, and then hold those weights fixed in **MCGatherer<Array<double,1> >** by calling the member function

```
void fix_weights(int target,
                 const Array<int,1>& CV,
                 const Array<double,1>& CV_expectation)
```

This resets the variables collecting the simulation results to zero, discarding the simulations used to calculate the weights. This means that the Monte Carlo estimates using the fixed weights will be calculated based on a set of simulations that are independent of those used to determine the weights, thus removing this source of potential bias. Furthermore, **fix_weights()** sets a flag **weights_fixed**, which causes the accumulation operator **+=** to calculate each simulated value of the target payoff by

$$f(u_i) - \sum_{j=1}^J \beta_j (g_j(u_i) - E[g_j(U)])$$

which is now possible because the weights β_j are fixed.

Consequently, **mean(target)** will now return the Monte Carlo estimate for $E[f(U)]$ using the control variates (as opposed to the “plain” Monte Carlo estimate when **weights_fixed** is not set) and **stddev(target)** will return the corresponding standard deviation.

The file **CVTestPower.cpp** on the website demonstrates this two-staged approach for variance reduction with multiple control variates in the case of a power option with payoff

$$\max(0, S(T)^\alpha - K) \tag{7.66}$$

where five standard options with different strikes are used as control variates. Table 7.4 illustrates the resulting variance reduction: By using five control variates, the accuracy of the Monte Carlo estimate is improved by a factor of about 23. Since this requires six times as many payoff evaluations as plain Monte Carlo, computational efficiency is improved by a factor of about 3.8. It is also worth noting that the regression (i.e. solving (7.57)) to determine the optimal weights is equivalent to approximating the power option payoff by a linear combination (i.e. portfolio) of the payoffs of the standard options used as

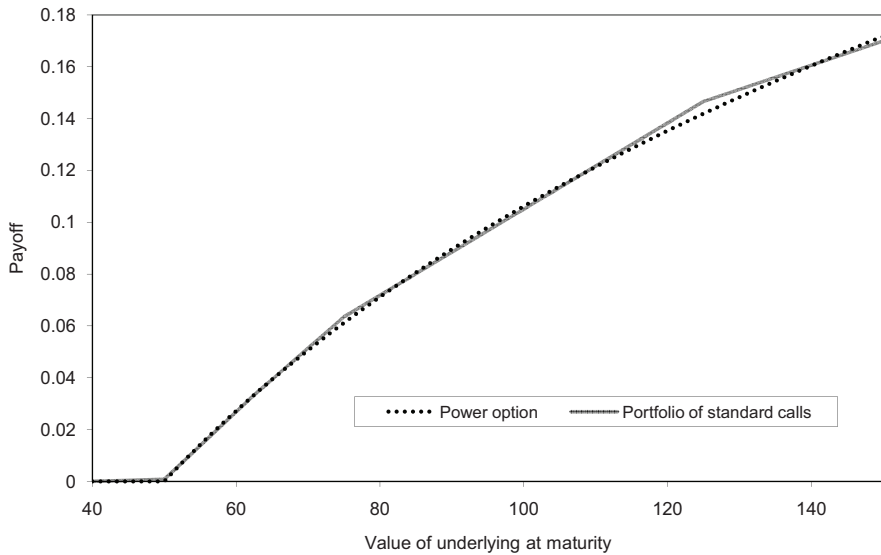


Figure 7.3 *Power option payoff approximated by payoff of a portfolio of five standard call options, based on the optimal control variate weights*

control variates: Figure 7.3 plots the payoff of the power option and the payoff of a portfolio of the standard options, where the portfolio weights are given by the (estimated) optimal control variate weights $\hat{\beta}_N$. Clearly, whenever a target payoff can be closely approximated by a portfolio of control variate payoffs, the portfolio and the target will be closely correlated, and the portfolio will be an effective control variate. Conversely, as Figure 7.3 illustrates, the regression (7.57) will converge for the portfolio payoff to optimally (in a probability-weighted least squares sense) approximate the target payoff.

7.7 Pricing instruments with early exercise features

At each point in time at which an option permits early exercise, the value of the option is given by the greater of the early exercise payoff and the *continuation value* (i.e. the value of not exercising early and instead holding on to the option). When using binomial lattice models (Chapter 3) or finite difference schemes (Chapter 5), one can price the option by backward induction, and thus the continuation value is known whenever the early decision needs to be evaluated. When simulating a Monte Carlo path, the continuation value is *not* known — herein lies the difficulty of pricing American and Bermudan options by Monte Carlo methods. However, in high-dimensional problems — be it due to a large number of underlying assets or due to the nature of the stochastic model — Monte Carlo simulation remains the primary numerical method.

Also, as we will see below, once we are able to determine (an approximation for) the continuation value, we are in a position to also price path dependent options with early exercise features (including barrier options, Asian options and lookback options) in a generic manner that is straightforward to implement.

7.7.1 *The regression-based algorithm*

The most common approach to approximate the continuation value is to estimate it by regression on a set of basis functions of state variables on a Monte Carlo path. Methods of this type have been proposed by a number of authors, including Carrière (1996), Tsitsiklis and van Roy (1999) and Longstaff and Schwartz (2001). Given a Monte Carlo path, for each time point where early exercise is permitted we can look ahead and see what the value of holding on to the option would be – at the last early exercise time prior to maturity this is simply the discounted payoff at maturity on that path. However, at the time the option holder must make a decision whether or not to exercise early, only information on the path up to that point in time is available, summarised by a set of state variables.²³ Thus we need a function which predicts the continuation value based on the realisation of the state variables. It is obtained by choosing a set of basis functions of the state variables, simulating a set of Monte Carlo paths (the *training paths*), and on these paths performing a linear regression of the realised discounted value of holding on to the option regressed on the values of the basis functions at the time of potential early exercise. Once the coefficients of this regression have been determined and we thus have a function which maps the values of the state variables at the time of potential early exercise to a (predicted) continuation value, this continuation value can be compared to the early exercise payoff in order to decide whether to exercise or not. This early exercise strategy is then used on a fresh (i.e. independent) set of Monte Carlo paths to yield an estimate for the option value.

As the early exercise strategy only uses information that would be available to the option holder at the time of exercise, it can be no better than the best possible (i.e. optimal) strategy based on available information. In other words, the approximation of the continuation value by regression on state variables results in a suboptimal exercise strategy, and the Monte Carlo estimator — in the absence of any other bias — will converge to a lower bound on the value of the option. The quality of the lower bound depends on how well the regression on the chosen set of basis functions approximates the conditional expectation of the discounted value of not exercising the option early — thus a better set of basis functions yields a higher option value. One should note that Longstaff and Schwartz (2001) use the same set of paths to conduct the regression and

²³ Depending on the model and the (potentially path-dependent) payoff, the set of state variables may be quite large — in some cases it might include the entire history up to that point in time.

the calculation of the Monte Carlo estimator for the option value. In this case one can no longer make a definitive statement about the direction of the bias, though in practice it appears that the estimator is typically still biased low.

When reliable error bounds are required, it is better to ensure that the regression-based method described here converges to a lower bound: The correct value of an option with the possibility of early exercise can be written in terms of expectations under a risk-neutral measure

$$\sup_{\tau} E[h_{\tau}(X_{\tau})] \quad (7.67)$$

where X_{τ} is the set of state variable at time τ , $h_{\tau}(X_{\tau})$ is the discounted payoff resulting from exercise at time τ , and τ is a stopping time. If the decision to exercise at time $\hat{\tau}$ is based on a continuation value function estimated by regression on an independent (i.e. separate) set of training paths, $\hat{\tau}$ is a stopping time and the Monte Carlo estimator converges to $E[h_{\hat{\tau}}(X_{\hat{\tau}})]$ with

$$E[h_{\hat{\tau}}(X_{\hat{\tau}})] \leq \sup_{\tau} E[h_{\tau}(X_{\tau})] \quad (7.68)$$

One could then make use of a dual formulation (established by Haugh and Kogan (2004) and Rogers (2002)) of the pricing problem to obtain an upper bound, given by

$$\sup_{\tau} E[h_{\tau}(X_{\tau})] \leq \inf_M E[\max_{k=1, \dots, m} (h_{t_k}(X_{t_k}) - M_{t_k})] \quad (7.69)$$

where the infimum is taken over martingales M under the risk-neutral measure, with initial value 0. The t_k , $k = 1, \dots, m$, are the possible exercise times.

For $M_t \equiv 0 \forall t$, the expectation on the right-hand side of (7.69) corresponds to early exercise with perfect foresight, which is clearly an upper bound for the value of the option. However, this is not a very good upper bound, and the quality of the upper bound hinges on a good choice of the martingale M . One attractive, computationally tractable choice of M has been proposed by Belomestny, Bender and Schoenmakers (2009).

Many practical applications restrict themselves to the lower-bound, regression-based method, and this is the approach that we will take here. Consider a set of financial variables²⁴ X_i , $i = 1, \dots, N$, observed at times T_k , $k = 0, \dots, K$. Furthermore, denote by $T_{\bar{k}(j)}$, $0 \leq \bar{k}(j) \leq K$ the j -th time of potential exercise of an option (i.e. the option holder does not necessarily have the right of early exercise at *all* times T_k). For each early exercise time $T_{\bar{k}(j)}$, $0 \leq \bar{k}(j) < K$, we will approximate the continuation value by a linear combination of n basis

²⁴ These financial variables may or may not be assets as in Chapter 4.

functions,

$$V(T_{\bar{k}(j)}) = \sum_{\ell=1}^n c_{\ell j} \psi_{\ell j} \quad (7.70)$$

$$\psi_{\ell j} := \psi_{\ell} \left(T_{\bar{k}(j)}, \{X_i(T_k)\}_{\substack{i=1, \dots, N, \\ k=0, \dots, \bar{k}(j)}} \right), \quad \ell = 1, \dots, n$$

which is to say that the $\psi_{\ell j}$ can, in general, be functions of time and the entire history of observed financial variables until time $T_{\bar{k}(j)}$. Following Longstaff and Schwartz (2001), the algorithm to estimate $c_{\ell j}$ for each $T_{\bar{k}(j)}$ proceeds like this:

Step 1: Generate `npaths` simulated Monte Carlo paths of the financial variables X_i observed at times T_k . (These paths will be used to estimate the $c_{\ell j}$ and then discarded.)

Step 2: Initialise a vector y of length `npaths` with the option payoff at time T_K on each path, discounted to the *last* time point $T_{\bar{k}(J)} < T_K$ of potential early exercise. Set the “current” time $t = T_{\bar{k}(J)}$. The initial value of the index j is J (i.e. we will be proceeding by backward induction).

Step 3: Calculate a vector e of length `npaths`, containing the payoff on each path if the option is exercised at time t .

Step 4: Determine the $c_{\ell j}$ for all $\ell = 1, \dots, n$ and the current j by least squares fit, i.e. choose the $c_{\ell, j}$ to minimise

$$\sum_{h=1}^{\text{npaths}} (y_h - V^{(h)}(T_{\bar{k}(j)}))^2 \quad (7.71)$$

where $V^{(h)}(T_{\bar{k}(j)})$ is given by (7.70) calculated using the realisations of $X_i(T_k)$ on the h -th path.

Step 5: Using the thus determined $c_{\ell j}$, set each y_h to $\max(e_h, V^{(h)}(T_{\bar{k}(j)}))$ discounted to time $T_{\bar{k}(j-1)}$.

Step 6: If $T_{\bar{k}(j)}$ is the earliest time of potential early exercise, stop. Otherwise, set $j = j - 1$ and $t = T_{\bar{k}(j)}$, and go to Step 3.

Longstaff and Schwartz (2001) suggest that in the objective function (7.71), one may want to omit those paths for which e_h at the “current” time t is zero.

7.7.2 Implementation

As noted in the previous section, in order to guarantee that the Monte Carlo price for an option with early exercise features can be interpreted as a lower bound on the true price (as opposed to having uncertainty about the direction of the bias), it is recommended to conduct the procedure in two independent steps. First, the multivariate regression approximating the early exercise boundary is carried out on a set of simulated asset price paths (call these the *training paths*). The Monte Carlo price is then calculated by applying

```

Array<double,2> paths(8,4);
paths = 1.00, 1.09, 1.08, 1.34,
        1.00, 1.16, 1.26, 1.54,
        1.00, 1.22, 1.07, 1.03,
        1.00, 0.93, 0.97, 0.92,
        1.00, 1.11, 1.56, 1.52,
        1.00, 0.76, 0.77, 0.90,
        1.00, 0.92, 0.84, 1.01,
        1.00, 0.88, 1.22, 1.34;
Array<double,1> T(4);
T = 0.0, 1.0, 2.0, 3.0;
double K = 1.10;
Payoff put(K,-1);
boost::function<double (double)> f;
f = boost::bind(std::mem_fun(&Payoff::operator()),&put,_1);
FlatTermStructure ts(0.06,0.0,10.0);
LongstaffSchwartzExerciseBoundary1D ls(T,ts,paths,f,2);
MCGatherer<double> MCestimate;
Array<double,1> path(4);
for (i=0;i<8;i++) {
    path = paths(i,Range::all());
    MCestimate += ls.apply(path); }
std::cout << MCestimate.mean() << std::endl;

```

Listing 7.18: Longstaff/Schwartz toy example using LongstaffSchwartz-ExerciseBoundary1D

the (suboptimal, because approximate) early exercise strategy to an independent set of paths. This separation suggests an approach to implementation: The first step can be implemented by constructing an object representing the exercise boundary from the training paths, and the second step involves creating an `MCMapping` from this exercise boundary, which can then be fed into the generic Monte Carlo algorithm `MCGeneric`.

Let us consider first the C++ class representing the exercise boundary. In the previous section, the continuation value at any time t could potentially depend on several financial variables observed at a number of time points up to and including time t , and the most general implementation of the exercise boundary needs to reflect this.²⁵ The class `RegressionExerciseBoundary` provides this functionality, but for clarity of exposition consider first the simpler case where again following Longstaff and Schwartz (2001) the continuation value is approximated by a polynomial function of the value of a single underlying

²⁵ Note that this may be needed regardless of whether the dynamics of the financial variables are Markovian, because the option payoff itself may be path dependent — for example if one had an Asian option that can be exercised early.

financial variable at time $T_{\bar{k}(j)}$. In this case (7.70) becomes, for some fixed i ,

$$V(T_{\bar{k}(j)}) = \sum_{\ell=1}^n c_{\ell j} (X_i(T_{\bar{k}(j)}))^{\ell-1} \quad (7.72)$$

This is implemented in the class `LongstaffSchwartzExerciseBoundary1D`.²⁶ The regression to estimate the coefficients $c_{\ell j}$ is carried out in the constructor, which takes as arguments the timeline of time points of potential exercise, a (number of paths) $\times$ (number of time points) `Array` of “training paths,” an initial `TermStructure` (for discounting), a `boost::function<double (double)>` to calculate the (undiscounted) payoff at time of exercise as a function of the underlying financial variable, and the degree $(n-1)$ of the polynomial (7.72). The constructor also has a Boolean optional argument, which if set to `true` causes the regression at time point $T_{\bar{k}(j)}$ to be applied only to those paths for which the early exercise payoff at $T_{\bar{k}(j)}$ is positive (the default is `true`).

Once the early exercise boundary is constructed, the resulting early exercise strategy (to exercise the option if and only if the early exercise payoff is greater than the approximate continuation value given by (7.72)) can be applied to each Monte Carlo path to obtain the discounted payoff on that path. This is implemented in the member function `apply()` of `LongstaffSchwartzExerciseBoundary1D`.²⁷ Using `LongstaffSchwartzExerciseBoundary1D`, one can now reproduce the toy example of Longstaff and Schwartz (2001) of an American put option valued on eight paths with four time points by the code in Listing 7.18.²⁸

The class `LongstaffSchwartzExerciseBoundary` generalises `LongstaffSchwartzExerciseBoundary1D` to allow for multiple state variables (but still only observed at the time of potential early exercise) and arbitrary basis functions (rather than polynomials) in the regression (7.70), as well as discounting by an arbitrary numeraire (rather than deterministic discounting using an initial `TermStructure`). Thus the arguments of the constructor are

- an `Array<double,1>` containing the time points of potential exercise (the timeline)
- a (number of paths) $\times$ (number of time points) $\times$ (number of underlying financial variables) `Array<double,3>` of training paths
- a (number of paths) $\times$ (number of time points) `Array<double,2>` containing the realisations of numeraire values on each training path
- a `boost::function<double (double,const Array<double,1>&>` functor to calculate the (undiscounted) payoff at time of exercise t as a function

²⁶ See `LongstaffSchwartz.hpp` and `LongstaffSchwartz.cpp` on the website.

²⁷ Note that `apply()` assumes (but does not check) that the timeline corresponding to the Monte Carlo path (represented by `Array<double,1>`) is the same as the timeline of times of potential early exercise for which the training paths were supplied to the constructor.

²⁸ This is an excerpt from `LS_test.cpp`, available in full on the website. Note that Longstaff/Schwartz reuse the “training paths” to obtain a Monte Carlo estimate for the price. As stated previously, if one wants to ensure that the Monte Carlo estimate converges to a lower bound for the price, one should not reuse paths in this way.

```

Array<double,2> paths(8,4);
paths = 1.00, 1.09, 1.08, 1.34,
        1.00, 1.16, 1.26, 1.54,
        1.00, 1.22, 1.07, 1.03,
        1.00, 0.93, 0.97, 0.92,
        1.00, 1.11, 1.56, 1.52,
        1.00, 0.76, 0.77, 0.90,
        1.00, 0.92, 0.84, 1.01,
        1.00, 0.88, 1.22, 1.34;
Array<double,1> T(4);
T = 0.0, 1.0, 2.0, 3.0;
double K = 1.10;
Payoff put(K,-1);
boost::function<double (double)> f;
f = boost::bind(std::mem_fun(&Payoff::operator()),&put,_1);
FlatTermStructure ts(0.06,0.0,10.0);
Array<double,3> genpaths(8,4,1);
genpaths(Range::all(),Range::all(),0) = paths;
Array<double,2> numeraire_values(8,4);
for (i=0;i<4;i++) numeraire_values(Range::all(),i) = 1.0/ts(T(i));
boost::function<double (double,const Array<double,1>&)>
    payoff = boost::bind(LSArrayAdapter,_1,_2,f,0);
vector<boost::function<double (double,const Array<double,1>&)> >
    basisfunctions;
int degree = 2;
Array<int,1> p(1);
for (i=0;i<=degree;i++) {
    p(0) = i;
    add_polynomial_basis_function(basisfunctions,p); }
LongstaffSchwartzExerciseBoundary
genls(T,genpaths,numeraire_values,payoff,basisfunctions);
Array<double,2> genpath(4,1);
Array<double,1> num(4);
for (i=0;i<8;i++) {
    genpath = genpaths(i,Range::all(),Range::all());
    num = numeraire_values(i,Range::all());
    MCEstimate += genls.apply(genpath,num); }
std::cout << MCEstimate.mean() << std::endl;

```

Listing 7.19: Longstaff/Schwartz toy example using LongstaffSchwartz-ExerciseBoundary

of t and the underlying financial variables at time t (contained in the `Array finvars`)

- a `std::vector` of

```

boost::function<double(double t,
                        const Array<double,1>& finvars)

```

functors representing the basis functions ψ_ℓ in (7.70), where in this case

$$\psi_{\ell j} := \psi_\ell(T_{\bar{k}(j)}, \{X_i(T_{\bar{k}(j)})\}_{i=1,\dots,N}), \quad \ell = 1, \dots, n$$

is a function of time and the underlying financial variables observed at times of potential early exercise $T_{\bar{k}(j)}$ *only*.

- an optional Boolean flag as in `LongstaffSchwartzExerciseBoundary1D`

Note that the basic structure of the regression algorithm in the constructor and the application of the early exercise strategy to Monte Carlo paths through the member function `apply()` remains the same,²⁹ where `apply()` now takes as arguments

- a (number of time points) $\times$ (number of underlying financial variables) `Array<double,2>` representing a Monte Carlo path
- a (number of time points) `Array<double,1>` of numeraire values on that path

Listing 7.19 reproduces the toy example from Longstaff and Schwartz (2001) using `LongstaffSchwartzExerciseBoundary`.³⁰

The class `RegressionExerciseBoundary` implements the regression (7.70) in its full generality. Compared to `LongstaffSchwartzExerciseBoundary`, the only arguments of the constructor that change are

- the functor to calculate the (undiscounted) payoff at time of exercise t : This is now a

```
boost::function<double (const Array<double,1>& t_history,
                        const Array<double,2>& finvars)>,
```

where `finvars` is a (number of underlying financial variables) $\times$ (number of observation time points up to t) `Array` of values of the underlying financial variables on a given path, observed at the time points in the (number of observation time points up to t) `Array` `t_history`

- the `std::vector` of functors representing the basis functions $\psi_\ell(\cdot, \cdot)$ in (7.70), which are now of the type

```
boost::function<double (const Array<double,1>& t_history,
                        const Array<double,2>& finvars)>,
```

analogously to the payoff functor.

Similarly, the arguments of the member function `apply()` do not change. In `LS_Test.cpp` on the website, the toy example is also reproduced using

²⁹ See `LongstaffSchwartz.cpp` on the website.

³⁰ This is an excerpt from `LS_Test.cpp`, available in full on the website. The utility functions `add_polynomial_basis_function()` (to create the appropriate functor representing a polynomial) and `LSArrayAdapter()` (to convert a `boost::function<double (double)>` into the corresponding `boost::function<double (double, const Array<double,1>&)>`) are implemented in `LongstaffSchwartz.cpp`.

RegressionExerciseBoundary.³¹ One should note that **RegressionExerciseBoundary** assumes that the history of observations of the underlying financial variables relevant for the evaluation of payoffs and the basis functions approximating the continuation value is limited to time points of potential early exercise. This assumption may be too restrictive when dealing with Bermudan options (with infrequent early exercise opportunities) on path-dependent payoffs.

Moving beyond the toy example, one can proceed to implement the actual Monte Carlo simulation of American options. All that changes is that the “paths” of the toy example are replaced by a (typically much larger) set of randomly generated paths of the underlying price process and the numeraire. To assist with this, we implement the class template **MCTrainingPaths**³², which creates a $\text{paths} \times \text{timepoints} \times (\text{number of state variables})$ **Array** of state variable realisations and the corresponding $\text{paths} \times \text{timepoints}$ **Array** of numeraire values, as required by the constructors of **LongstaffSchwartzExerciseBoundary** and **RegressionExerciseBoundary**.

Listing 7.20 illustrates the construction of a **LongstaffSchwartzExerciseBoundary** for an American put option. The template parameters of **MCTrainingPaths** are a class representing the underlying stochastic process (in this case **GeometricBrownianMotion**) and a class for generating the random numbers to drive the stochastic process (in this case the pseudo-random

```
RandomArray<ranlib::NormalUnit<double>,double>
```

— an alternative would be the quasi-random **SobolArrayNormal**). The class representing the stochastic process must implement

- a member function **dimension()**, which returns the number of state variables (i.e. the dimension) of the process,
- a member function **set_timeline(const Array<double,1>& timeline)**, which sets the time points for which state variable (and numeraire) realisations will be generated,
- an

```
operator()(Array<double,2>& underlying_values,
           const random_variable& x,
           const TermStructure& ts)
```

and an

³¹ The utility functions **add_polynomial_basis_function()** (to create the appropriate functor representing a polynomial) and **REBAdapter()** (to convert a **boost::function<double(double)>** into the corresponding **boost::function<double(const Array<double,1>&,const Array<double,2>&)** are implemented in **LongstaffSchwartz.cpp**.

³² See **MCAmerican.hpp** on the website.

```

BlackScholesAsset stock(&vol,S);
std::vector<const BlackScholesAsset*> underlying;
underlying.push_back(&stock);
FlatTermStructure ts(r,0.0,mat+10.0);
exotics::StandardOption Mput(stock,T(0),mat,K,ts,-1);
GeometricBrownianMotion gbm(underlying);
gbm.set_timeline(T);
ranlib::NormalUnit<double> normalRNG;
RandomArray<ranlib::NormalUnit<double>,double>
    random_container(normalRNG,gbm.factors(),gbm.number_of_steps());
MCTrainingPaths<GeometricBrownianMotion,
    RandomArray<ranlib::NormalUnit<double>,double> >
    training_paths(gbm,T,train,random_container,ts,numeraire_index);
Payoff put(K,-1);
boost::function<double (double)> f;
f = boost::bind(std::mem_fun(&Payoff::operator()),&put,_1);
boost::function<double (double,const Array<double,1>&)> payoff =
    boost::bind(LSArrayAdapter,_1,_2,f,0);
std::vector<boost::function<double (double,
                                const Array<double,1>&)> >
    basisfunctions;
Array<int,1> p(1);
for (i=0;i<=degree;i++) {
    p(0) = i;
    add_polynomial_basis_function(basisfunctions,p); }
boost::function<double (double,double)> put_option;
put_option =
    boost::bind(&exotics::StandardOption::price,&Mput,_1,_2);
boost::function<double (double,const Array<double,1>&)>
put_option_basis_function =
    boost::bind(LSArrayAdapterT,_1,_2,put_option,0);
if (include_put)
    basisfunctions.push_back(put_option_basis_function);
LongstaffSchwartzExerciseBoundary
    boundary(T,training_paths.state_variables(),
        training_paths.numeraires(),payoff,basisfunctions);

```

Listing 7.20: Construction of a LongstaffSchwartzExerciseBoundary for an American put option

```

operator() (Array<double,2>& underlying-values,
            const random_variable& x,
            const TermStructure& ts,
            int numeraire_index)

```

which generate the realisations of the stochastic process at the dates in the required timeline using a draw x of the random variable (the type of which must be consistent with the chosen class for generating the random

```

LSEExerciseStrategy<LongstaffSchwartzExerciseBoundary>
    exercise_strategy(boundary);
MCMMapping<GeometricBrownianMotion,Array<double,2> >
    mc_mapping(exercise_strategy,gbm,ts,numeraire_index);
boost::function<double (Array<double,2>)> func =
    boost::bind(&MCMMapping<GeometricBrownianMotion,
        Array<double,2> >::mapping,
            &mc_mapping,_1);
MCGeneric<Array<double,2>,double,
    RandomArray<ranlib::NormalUnit<double>,double> >
    mc(func,random_container);
MCGatherer<double> mcgatherer;
size_t n = minpaths;
boost::math::normal normal;
double d = boost::math::quantile(normal,0.95);
cout << "Paths,MC value,95% CI lower bound,95% CI upper bound"
    << endl;
while (mcgatherer.number_of_simulations()<maxpaths) {
    mc.simulate(mcgatherer,n);
    cout << mcgatherer.number_of_simulations() << ', '
        << mcgatherer.mean() << ', '
        << mcgatherer.mean()-d*mcgatherer.stddev() << ', '
        << mcgatherer.mean()+d*mcgatherer.stddev() << endl;
    n = mcgatherer.number_of_simulations(); }

```

Listing 7.21: Pricing an American put option using LongstaffSchwartzExerciseBoundary and the generic Monte Carlo algorithm implementation

numbers — typically, x will be an `Array<double,2>`). The first version of the `operator()` generates the realisations under the martingale measure associated with taking the savings account as the numeraire (corresponding to deterministic discounting in the present example), while the second version uses the martingale measure associated with a chosen numeraire — in the case of `GeometricBrownianMotion`, the variable `numeraire_index` allows us to choose one of the assets as the numeraire.

The constructor of `MCTrainingPaths` then takes as arguments

- an instance of the class representing the underlying stochastic process
- the set of time points for which realisations of the stochastic process are to be generated
- the number of paths to be generated
- an instance of the random variable generator
- the initial `TermStructure`
- the `numeraire_index`

The thus constructed instance of `MCTrainingPaths` is then used to instantiate a `LongstaffSchwartzExerciseBoundary` via

Paths	Polynomial only			Incl. European put		
	MC price	95% conf. bounds		MC price	95% conf. bounds	
10240	12.357	12.1589	12.5551	12.4288	12.2224	12.6352
20480	12.3821	12.2416	12.5225	12.4645	12.3181	12.6109
40960	12.472	12.3725	12.5715	12.538	12.4343	12.6417
81920	12.4689	12.3986	12.5392	12.5196	12.4464	12.5929
163840	12.4582	12.4086	12.5079	12.5054	12.4537	12.5571
327680	12.4808	12.4457	12.516	12.5312	12.4945	12.5678
655360	12.5039	12.4791	12.5288	12.5525	12.5266	12.5784
1310720	12.4976	12.4801	12.5152	12.5442	12.5259	12.5625

Table 7.5 *Monte Carlo estimates for the price of an American put using LongstaffSchwartzExerciseBoundary, polynomial basis functions up to degree 3, with and without the European put with the same maturity included as one of the basis functions. Further inputs: initial stock price 100, strike 102, continuously compounded interest rate 5%, time to maturity 1.5 years, volatility 30%, no dividends. Time line refinement: 10. Number of “training paths” to estimate exercise boundary: 100,000.*

```
LongstaffSchwartzExerciseBoundary
    boundary(T,training_paths.state_variables(),
             training_paths.numeraires(),
             payoff,basisfunctions);
```

(7.73)

In order to apply the generic Monte Carlo algorithm implementation, we need to create a class derived from `MCPayoff`, which calculates the payoff on each path using the member function `apply()` of the exercise boundary. This is implemented as the

```
template<class exercise_boundary>
class LSEExerciseStrategy : public MCPayoff
```

(7.74)

Instantiating

```
LSEExerciseStrategy<LongstaffSchwartzExerciseBoundary>
exercise_strategy(boundary);
```

(7.75)

supplies the `MCPayoff` that allows us to instantiate first an `MCMapping` and then an `MCGeneric` object to execute the Monte Carlo simulation as before — see Listing 7.21.³³ Note that if the flag `include_put` evaluates to true in Listing 7.20, the price (at each time of potential early exercise) of the European put expiring at the same time as the American put is included as one of the basis functions. One would expect that this would improve the regression fit of the exercise boundary, since if one ignores the value of subsequent intermediate exercise, the price of the European put can serve as a proxy for

³³ The full working code containing Listings 7.20 and 7.21 is given in `LSMC.cpp` on the website. For further illustration, `LSMCpath.cpp` on the website replaces `LongstaffSchwartzExerciseBoundary` with the more general `RegressionExerciseBoundary`.

No dividends						
	Polynomial only			Incl. European option		
Paths	MC price	95% conf. bounds		MC price	95% conf. bounds	
102400	23.7961	23.6361	23.9561	23.785	23.6273	23.9426
204800	23.8334	23.7203	23.9465	23.843	23.7314	23.9546
409600	23.8937	23.8135	23.9739	23.8906	23.8115	23.9697
819200	23.8621	23.8054	23.9189	23.8585	23.8025	23.9144
1638400	23.8456	23.8054	23.8857	23.8395	23.7999	23.8791
3276800	23.829	23.8006	23.8574	23.8295	23.8015	23.8575
6553600	23.8445	23.8244	23.8645	23.8404	23.8206	23.8602
13107200	23.859	23.8448	23.8732	23.8596	23.8456	23.8736
With dividends						
	Polynomial only			Incl. European option		
Paths	MC price	95% conf. bounds		MC price	95% conf. bounds	
102400	17.8089	17.696	17.9218	17.9196	17.8091	18.0301
204800	17.818	17.7382	17.8977	17.9389	17.8607	18.017
409600	17.8567	17.8002	17.9132	17.9679	17.9125	18.0232
819200	17.8272	17.7872	17.8672	17.9509	17.9117	17.9901
1638400	17.8288	17.8005	17.8571	17.949	17.9213	17.9767
3276800	17.8182	17.7982	17.8382	17.9456	17.926	17.9652
6553600	17.8266	17.8125	17.8408	17.953	17.9392	17.9669
13107200	17.8348	17.8248	17.8448	17.9609	17.9511	17.9707

Table 7.6 *Monte Carlo estimates for the price of an American Margrabe option using RegressionExerciseBoundary, polynomial basis functions up to degree 2, with and without the European Margrabe option with the same maturity included as one of the basis functions. Further inputs: initial stock prices 100 and 105, strike factor 1.02 (i.e. strike = 1.02 · 100/105), continuously compounded interest rate 5%, time to maturity 2.5 years, volatilities 30% and 45%, instantaneous correlation between asset dynamics 0.5, continuously compounded dividend yields 7% and 2%. Time line refinement: 10. Number of “training paths” to estimate exercise boundary: 100,000.*

the “continuation value” of the American put. Numerical experiments bear this out: For a sufficiently large number of simulations, Table 7.5 shows the Monte Carlo price for the American put to be significantly higher when the European put is included as one of the basis functions.

In general it makes sense to include the price of the option without early exercise (when this is available in closed form) as one of the basis functions for the regression of the continuation value. `LSMCMargrabe.cpp` on the website implements this for the option to exchange one (risky) asset for another — all

that changes relative to case of a plain American put is the payoff specification and a different set of basis functions, including the closed-form price of a European Margrabe option. Table 7.6 shows some illustrative numerical results. One should note that in the case of zero dividends on both assets, a result along the lines of Merton (1973b) holds, that it is never optimal to exercise the option early. The fact that in this case the lower bound on the option price obtained using Monte Carlo is less than the European option price reflects the sub-optimality of the exercise strategy obtained using the regression estimate of the exercise boundary. Setting the continuously compounded dividend yield of the asset obtained upon exercise to 7% and the dividend yield of the asset surrendered upon exercise to 2% results in a significant early exercise premium, and a significant improvement of the Monte Carlo price when using the European option price as one of the basis functions, as Table 7.6 shows.

7.8 Quasi-random Monte Carlo

Recall the formulation of the integration problem (7.7), which Monte Carlo methods approximate by

$$\int_{[0,1]^d} f(x)dx \approx \frac{1}{N} \sum_{i=1}^N f(u_i), \quad u_i \in [0,1]^d \quad (7.76)$$

For pseudo-random Monte Carlo,³⁴ the u_i represent independent draws from a uniform distribution on the unit hypercube, and we have seen that the pseudo-random Monte Carlo estimate converges with $\mathcal{O}(N^{-\frac{1}{2}})$. Quasi-random Monte Carlo seeks to improve on this convergence by a sequence of u_i that is deterministically chosen to fill the unit hypercube “more evenly” (in a sense to be made precise) than a random or pseudo-random sequence. This can result in an acceleration of convergence to nearly $\mathcal{O}(N^{-1})$ — herein lies the difference to the variance reduction techniques considered in Section 7.6, which only affected the implicit constant in $\mathcal{O}(N^{-\frac{1}{2}})$ convergence.

It is important to note that quasi-random Monte Carlo explicitly depends on the problem dimension. In pseudo-random Monte Carlo a single univariate random number generator was sufficient to generate all the u_i in (7.76), regardless of the dimension d , as all the components of the random vectors u_i are independent random variables, and the pseudo-random generators mimic independent random draws. In quasi-random Monte Carlo, on the other hand, two consecutive elements of the quasi-random sequence are not independent. Consequently, one must distinguish between the set of random variates making up a single random vector u_i required to produce a realisation of the simulation $f(u_i)$, and consecutive realisations of the random vector u_i drawn N times to calculate Monte Carlo estimate in (7.76): the dimension d of the

³⁴ The term “pseudo-random” as opposed to “random” acknowledges the fact that random number generators on a computer are still deterministic algorithms, but they produce sequences of numbers which are statistically indistinguishable from random sequences.

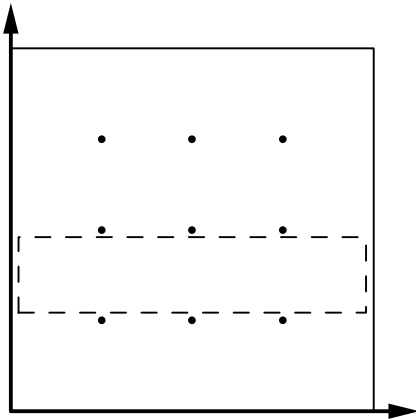


Figure 7.4 *A grid of points in the unit square: the dashed box is completely empty of points*

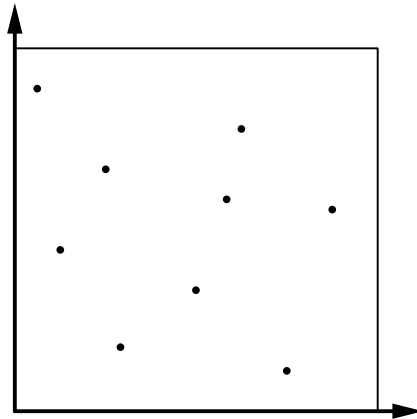


Figure 7.5 *Better coverage of the unit square*

u_i is the required dimension of the quasi-random sequence, while N is the length of the sequence.

For example, suppose one needs to generate 1,000 paths of geometric Brownian motion observed at 100 time steps, so N is 1,000 and d is 100. In pseudo-random Monte Carlo, one can generate 100,000 (i.e. $N \times d$) independent univariate uniform random variates, which are transformed into independent, normally distributed Brownian motion increments and then assembled (in batches of 100) to produce 1,000 paths. In quasi-random Monte Carlo, one has to generate a sequence of 1,000 quasi-random variates of dimension 100, i.e. one for each path — using 100,000 quasi-random variates of dimension 1 would lead to an incorrect result. As the rate of convergence of quasi-random methods tends to deteriorate in higher dimensions, it is recommended to keep the dimension d as low as possible, for example by using the predictor/corrector method whenever necessary and possible.

7.8.1 Generating quasi-random sequences

There is a large body of literature on quasi-random (also known as low-discrepancy) sequences, and the underlying theory can only be roughly sketched here. For a more in-depth treatment, Glasserman (2004) is a good starting point, who cites Niederreiter (1992) for the rigorous theoretical background.

In just one dimension, generating a sequence that covers the unit interval as uniformly as possible seems trivial — given N points, simply create a grid of points $1/(N + 1)$ apart. Even in two dimensions, things are no longer that simple. Consider Figure 7.4: the nine points are spaced in an even grid in the

unit square, but on each axis there is still empty space of length $1/4$ between points, and viewed in two dimensions there are large rectangles devoid of any points, like the one indicated in the figure. If the points are shifted as in Figure 7.5, these empty spaces are reduced, both in terms of two-dimensional rectangles and one-dimensional projections. In higher dimensions, a grid of points becomes even more problematic. Furthermore, in a grid the total number of points N has to be specified in advance and refining the grid by adding points becomes computationally very costly. Suppose we have points in a grid spaced $1/2^{k+1}$ apart in each of the d dimensions, i.e. a grid of 2^{kd} points. Refining the grid by adding points in the gaps along each dimension requires $2^{(k+1)d} - 2^{kd}$ additional points — this is exponential in d and grows very quickly in k . For appropriately chosen quasi-random sequences, on the other hand, much fewer additional points are needed to maintain uniformity on refinement.

Deviation from uniformity in quasi-random sequences is measured by discrepancy — thus the alternative nomenclature low-discrepancy sequences. These are defined relative to a collection $\mathcal{A}$ of (Lebesgue measurable) subsets of $[0, 1]^d$, with the discrepancy of a sequence $\{u_1, \dots, u_N\}$, $u_i \in [0, 1]^d$ given by

$$D(u_1, \dots, u_N; \mathcal{A}) = \sup_{A \in \mathcal{A}} \left| \frac{\#\{u_i \in A\}}{n} - \text{vol}(A) \right| \quad (7.77)$$

where $\#\{u_i \in A\}$ denotes the number of u_i contained in $A \in \mathcal{A}$ and $\text{vol}(A)$ is the volume (measure) of A . As Glasserman (2004) notes, the discrepancy is the supremum over errors in integrating the indicator function of A using the points $u_1, \dots, u_N$. Ordinary (also called extreme) discrepancy is obtained when $\mathcal{A}$ is the collection of all rectangles in $[0, 1]^d$ of the form

$$\prod_{j=1}^d [s_j, t_j), \quad 0 \leq s_j < t_j \leq 1 \quad (7.78)$$

When $\mathcal{A}$ is the collection of all rectangles

$$\prod_{j=1}^d [0, s_j), \quad 0 < s_j \leq 1 \quad (7.79)$$

one speaks of star discrepancy $D^*(u_1, \dots, u_N; \mathcal{A})$, which is a slightly weaker concept. Glasserman (2004) cites Niederreiter (1992) as stating that “it is widely believed” that in dimensions $d \geq 2$ there is a lower bound to star discrepancy in that any possible sequence $u_1, \dots, u_N$ will satisfy

$$D^*(u_1, \dots, u_N) \geq c_d \frac{(\ln N)^{d-1}}{N} \quad (7.80)$$

and the first n elements of any sequence $u_1, u_2, \dots$ will satisfy

$$D^*(u_1, \dots, u_n) \geq c'_d \frac{(\ln n)^d}{n} \quad (7.81)$$

for constants c_d, c'_d depending only on the dimension d . Since there are known

sequences which achieve the second lower bound, only such sequences are typically termed “low-discrepancy.” In the sense that discrepancy is the supremum over errors in integrating indicator functions, (7.81) represents an order of convergence. As Glasserman notes, the logarithmic term can be absorbed into any power of n , so this order becomes $\mathcal{O}(1/n^{1-\epsilon})$, for all $\epsilon > 0$.

There are a number of explicit constructions of such low-discrepancy sequences, but for finance applications there is a predominant focus on Sobol’ (1967) sequences. The reason for this is found in the concept of (t, m, d) -nets and (t, d) -sequences, as defined by Niederreiter (1987), which are a useful way of describing the qualities of a low-discrepancy sequence. Consider subsets of $[0, 1]^d$ of the form

$$\prod_{i=1}^d \left[\frac{a_i}{b^{j_i}}, \frac{a_i + 1}{b^{j_i}} \right), \quad (7.82)$$

for an integer $b \geq 2$ and with $j_i \in \{0, 1, \dots\}$ and $a_i \in \{0, 1, \dots, b^{j_i} - 1\}$.

This is called a b -ary box, also known as an *elementary interval* in base b . For integers $0 \leq t \leq m$, a (t, m, d) -net in base b is then defined as a set of b^m points in $[0, 1]^d$, such that exactly b^t points fall in each b -ary box of volume b^{t-m} . Therefore the fraction of points b^t/b^m that lie in each box equals the volume of that box. A (t, d) -sequence in base b is a sequence of points $u_1, u_2, \dots$ in $[0, 1]^d$ such that for all $m > t$ each segment $\{u_i : j b^m < i \leq (j+1)b^m\}$, $j = 0, 1, \dots$, is a (t, m, d) -net in base b .

The parameter t describes the uniformity of the net/sequence in the sense that t gives size of the b -ary boxes composing $[0, 1]^d$, which will contain exactly the right number of points — there is no statement made about uniformity within those boxes. A smaller t means smaller boxes and thus, loosely speaking, uniformity at a finer partition of $[0, 1]^d$. Note that (7.82) implies that the rectangles making up the partition can be any shape, from squares to narrow strips. Furthermore, (t, m, d) -nets and (t, d) -sequences in base b describe the uniformity of sets of b^m points, thus larger b means that a larger number of points is required for the uniformity properties to hold. Consequently, one would prefer a smaller base b , all other things being equal.

Taking into consideration both t and b , as well as computational efficiency in constructing the low-discrepancy sequence, leads one to favour Sobol’ (1967) sequences over those of, for example, Faure (1982). Sobol’ sequences are (t, d) -sequences in base 2 for all d , where t depends on d . Although Faure sequences improve on the latter, i.e. $t = 0$ for all d , they require a base at least as large as d , which becomes unworkable in the large dimensions often required in finance applications. Additionally, a base 2 has the computational advantage that Sobol’ sequences can be generated using fast, bit-level operations.

7.8.2 Sobol' sequence construction

Denote by $u_i \in [0, 1)^d$ the i -th point in a d -dimensional Sobol' sequence. The j -th component u_{ij} of the d -dimensional vector u_i is given by

$$u_{ij} = i_1 v_{1,j} \oplus i_2 v_{2,j} \oplus \cdots \quad (7.83)$$

where $\oplus$ is the bit-by-bit exclusive-or operator and i_k is the k -th binary digit of i , i.e.

$$i = \sum_k i_k 2^{k-1}$$

is the binary representation of i . The v_{kj} are the so-called *direction numbers* given by

$$v_{kj} := \frac{m_{kj}}{2^k} \quad (7.84)$$

$$m_{kj} := 2a_{1,j}m_{k-1,j} \oplus 2^2a_{2,j}m_{k-2,j} \oplus \cdots \oplus 2^{s_j-1}a_{s_j-1,j}m_{k-s_j+1,j} \\ \oplus 2^{s_j}m_{k-s_j,j} \oplus m_{k-s_j,j} \quad (7.85)$$

where the a_{hj} are either 0 or 1 and satisfy the condition that

$$x^{s_j} + a_{1,j}x^{s_j-1} + a_{2,j}x^{s_j-2} + \cdots + a_{s_j-1,j}x + 1 \quad (7.86)$$

is a primitive polynomial, i.e. is irreducible (cannot be factored) and the smallest p for which the polynomial divides $x^p + 1$ is $p = 2^{s_j} - 1$. In order to obtain Sobol' sequence, the initial values $m_{1,j}, m_{2,j}, \dots, m_{s_j,j}$ may be chosen arbitrarily provided that each $m_{k,j}$, $1 \leq k \leq s_j$, is odd and less than 2^k .

The quality of a Sobol' sequence depends a lot on the choice of direction numbers used to generate it. Joe and Kuo (2008) note that the error bounds for Sobol' sequences given by Sobol' (1967) indicate that one should use primitive polynomials in increasing order of their degrees, and leaving aside the special case for the first dimension of the Sobol' points where all the $m_{k,1}$ are 1, Joe and Kuo (2008) assign one primitive polynomial for each dimension starting from dimension 2. Thus they obtain a Sobol' sequence of dimension 1111 using all primitive polynomials up to degree 13, and dimension 21201 using all primitive polynomials up to degree 18. Subsequently, they seek to obtain "good" values for the first s_j values of m_{kj} required to initialise the recursion (7.85), in particular for Sobol' sequences in high dimensions. "Good" may be manifested by a small t (indicating higher uniformity) of a (t, d) -sequence, but in high dimensions the t of a Sobol' sequence cannot be determined practicably. Instead, Joe and Kuo (2008) proceed iteratively, where given the direction numbers for a Sobol' sequence in dimension $d - 1$, they select the numbers for dimension d to minimise the weighted maximum t -value of the two-dimensional projections of the first 2^m points in the sequence on the dimension (j, d) , $1 \leq j \leq d$, with higher dimensions j weighted lower by an arbitrary factor 0.9999^{j-1} and m varying between 1 and 31. A further adjustment is made to the search criterion to account for the fact that b^{t-m} is the volume of the b -ary boxes in the definition of a (t, m, d) -net, and thus accounting for the tradeoff between small t (fewer point required per box) and large $m - t$ (finer partition involving

smaller boxes). Since the number of possible sets of direction number grows exponentially in the dimension d , optimisation of the search criterion by a full search soon becomes infeasible, and Joe and Kuo (2008) deal with this by only considering a number of randomly generated sets of direction numbers when optimising based on the search criterion in dimensions higher than 19 (for $d \leq 19$, they conduct optimisation based on full enumeration of the sets of direction numbers).

Thus there are a lot of compromises made by Joe and Kuo in their search for “optimal” direction numbers, the most significant being the restriction of the search criterion to the t -values of the two-dimensional projections. However, these direction numbers appear to be the best currently available. With regard to their focus on two-dimensional projections, they note that “it is often the low-dimensional projections that are important in certain applications.” In many (possibly most) problems in quantitative finance, it appears that the integrand depends mostly on a small number of variables, or can be well approximated by a sum of functions with each depending on only a small number of variables at a time. The optimisation of Joe and Kuo favours the initial dimensions (by proceeding iteratively from dimension $d - 1$ to d and by assigning a lower weight 0.9999^{j-1} to higher dimensions j in the search criterion), but nevertheless it appears that the quality of the Sobol’ sequences generated with their direction numbers does not deteriorate substantially in higher dimensions, as opposed to Sobol’ sequences that had previously been in widespread use. Thus the approach championed by authors such as Jäckel (2002), to use the leading dimensions for the “most important” components of the problem, for example by simulating diffusions by Brownian Bridge techniques, no longer leads to such a significant improvement in convergence of (high-dimensional) quasi-random Monte Carlo prices — to the point that the improvement may no longer justify the computational effort.³⁵

7.8.3 Implementation of quasi-random Monte Carlo

The quasi-random sequence generator class `Sobol` in `QFQuasiRandom.hpp` is a wrapper for the C++ code implementing (7.83)–(7.85), modified and slightly optimised from the code provided by Joe and Kuo (2008) on their website.³⁶ The constructor

```
Sobol(int dimension, size_t maximum_number_of_points,
      const char* direction_number_file) (7.87)
```

takes as arguments the dimension d , the maximum number of points required in the sequence, and the name of a text file containing the initial direction number. The last argument defaults to “new-joe-kuo-6.21201.dat”, also provided by Joe and Kuo and containing direction numbers for up to 21201 dimensions. The member function

```
Array<double, 1>& random()
```

³⁵ The author is grateful to Karl Gellert for pointing this out.

³⁶ See <http://www.maths.unsw.edu.au/~fkuo/sobol/>

```

/* set up timeline - N is the number of time steps,
   mat is the maturity of the option */
Array<double,1> T(N+1);
firstIndex idx;
double dt = mat/N;
T = idx*dt;
// set up first asset - volatility level is sgm on both factors
Array<double,1> sgm1(2);
sgm1 = sgm;
ConstVol vol(sgm1);
BlackScholesAsset stock(&vol,S,0.03);
// vector containing underlying assets
std::vector<const BlackScholesAsset*> underlying;
underlying.push_back(&stock);
// term structure of interest rates
FlatTermStructure ts(r,0.0,mat+10.0);
// set up second asset
Array<double,1> sgm2(2);
sgm2 = 0.1, 0.4;
ConstVol vol2(sgm2);
BlackScholesAsset stock2(&vol2,S,0.04);
underlying.push_back(&stock2);
/* closed form price via formula implemented as member function
   of BlackScholesAsset - K/S is strike factor */
double margrabe_price = stock.Margrabe(stock2,mat,K/S);
cout << "Closed form Margrabe option: " << margrabe_price;

```

Listing 7.22: Closed-form solution for a Margrabe option to benchmark quasi-random Monte Carlo simulation

returns the next point in the sequence, and `void reset()` sets the state of the sequence generator back to the beginning.

Each draw in a typical Monte Carlo simulation in quantitative finance involves a number (say n) of independent, possibly multidimensional (say j -dimensional) random variables. Thus the dimension of the required Sobol' sequence is $j \times n$ and it may be convenient to rearrange the $(j \times n)$ -dimensional Sobol points into a $j \times n$ two-dimensional `Array<double,2>`. For "raw" Sobol points this is done by the class `SobolArray`, with constructor

```

SobolArray(int rows,int cols,
           size_t maximum_number_of_points,
           const char* direction_number_file)

```

(7.88)

where the member function `Array<double,2>& random()` returns the next element in the sequence. Instances of the class `SobolArrayNormal` are constructed in the same way, with the difference being that `random()` returns a reference to an `Array<double,2>` in which all numbers have been transformed by applying the inverse of the cumulative distribution function of the standard normal distribution, thus after multiplication by the square root of the time

```

// set up MBinaries
Array<int,2> xS(1,1);
xS = 1;
Array<int,2> mindex(2,2);
mindex = 0, 1,
          N, N;
Array<double,1> malpha(2);
malpha = 1.0, 0.0;
Array<double,2> mA(1,2);
mA = 1.0, -1.0;
Array<double,1> ma(1);
ma = K/S;
MBinary M1(underlying,ts,T,mindex,malphi,xS,mA,ma);
double M1price = M1.price();
Array<double,1> malphi2(2);
malphi2 = 0.0, 1.0;
MBinary M2(underlying,ts,T,mindex,malphi2,xS,mA,ma);
cout << "MBinary price of Margrabe option: "
      << M1price-K/S*M2.price() << endl;

```

Listing 7.23: Setting up M -binaries for a Margrabe option to benchmark quasi-random Monte Carlo simulation

step length $\sqrt{\Delta t}$ this $(j \times n)$ **Array** can be used to generate n increments of a j -dimensional Brownian motion.

Listings 7.22–7.24 demonstrate how to use quasi-random Monte Carlo to price a Margrabe option (i.e. option to exchange one asset for another, see Theorem 4.7) in a two-factor Black/Scholes model with N time steps, comparing the result to the closed-form solution.³⁷ The multiple time steps are for demonstration purposes only, as in this case of a European option a single time step would be sufficient. Note that the quasi-random Monte Carlo setup is essentially the same as the pseudo-random setup,³⁸ the only difference being that an instance of

RandomArray<ranlib::NormalUnit<double>,double> (7.89)

is replaced by an instance of **SobolArrayNormal**, and on a quasi-random simulation the standard deviation of the Monte Carlo estimate (as supplied by **MCGatherer::stddev()**) does not provide a meaningful error estimate.

7.8.4 Randomised quasi-random Monte Carlo

It is difficult to obtain a meaningful error estimate through quasi-random methods alone, but a particularly simple way of getting around this problem

³⁷ The full working code for this example is given in the file **QRMargrabeExampleReduced.cpp** on the website.

³⁸ Both quasi-random and pseudo-random Monte Carlo for this option are implemented in the file **QRMargrabeExample.cpp** on the website.


```

//---- Margrabe option price by quasi-random Monte Carlo ----
// quasi-random number generator
SobolArrayNormal sobol(2,N,maxpaths);
// asset price process
GeometricBrownianMotion gbm(underlying);
// create MCPayoffList from MBinaries
boost::shared_ptr<MBinaryPayoff> M1payoff = M1.get_payoff();
boost::shared_ptr<MBinaryPayoff> M2payoff = M2.get_payoff();
MCPayoffList mcpayofflist;
mcpayofflist.push_back(M1payoff);
mcpayofflist.push_back(M2payoff,-K/S);
cout << "Strike coefficient: " << K/S << endl;
/* MCMapping to map random numbers to asset price realisations
   to discounted payoffs */
MCMapping<GeometricBrownianMotion,Array<double,2> >
    mc_mapping(mcpayofflist,gbm,ts,numeraire_index);
// mapping functor
boost::function<double (Array<double,2>)> func =
    boost::bind(&MCMapping<GeometricBrownianMotion,
        Array<double,2> >::mapping,&mc_mapping,_1);
// generic Monte Carlo algorithm object
MCGeneric<Array<double,2>,double,SobolArrayNormal>
    mc_QR(func,sobol);
MCGatherer<double> mcgathererQR;

/* run quasi-random Monte Carlo simulation and compare to
   closed form value */
unsigned long n = minpaths;
cout << "Minimum number of paths: " << minpaths
    << "\nMaximum number of paths: " << maxpaths << endl;
cout << "Margrabe option\nPaths,Closed form value,";
cout << "QR MC value\n";
double CFprice = margrabe.price();
while (n<=maxpaths) {
    cout << mcgathererQR.number_of_simulations() << ', '
        << CFprice << ',,';
    // simulate
    mc_QR.simulate(mcgathererQR,n);
    // output Monte Carlo result
    cout << mcgathererQR.mean() << ', ' << endl;
    n = mcgathererQR.number_of_simulations(); }

```

Listing 7.24: Pricing a Margrabe option by quasi-random Monte Carlo simulation

```

template <class price_process,
          class random_variable,
          class QR_generator>
class RandomShiftQMCMMapping {
private:
    MCMMapping<price_process,random_variable> mc_mapping;
    QR_generator& QRgen;
public:
    /// Constructor.
    inline RandomShiftQMCMMapping(QR_generator& xQRgen,
                                   MCPayoff& xpayoff,
                                   price_process& xprocess,
                                   const TermStructure& xts,
                                   int xnumeraire_index = -1)
        : mc_mapping(xpayoff,xprocess,xts,xnumeraire_index),QRgen(xQRgen) { };
    /** Choose the numeraire asset for the equivalent martingale measure
        under which the simulation is carried out. */
    inline bool set_numeraire_index(int xnumeraire_index)
    { return mc_mapping.set_numeraire_index(xnumeraire_index); };
    /** The function mapping a realisation of the shift to the (often
        multidimensional) quasi-random variable x to a discounted payoff. */
    double mapping(random_variable x);
    /** The function mapping a realisation of the shift to the (often
        multidimensional) quasi-random variable x to multiple discounted
        payoffs. */
    Array<double,1> mappingArray(random_variable x);
};

```

Listing 7.25: Class template for randomised quasi-random Monte Carlo simulation

is to randomise (in a pseudo-random sense) over a set of quasi-random simulation runs.³⁹ A simple way a quasi-random sequence can be randomised is by shifting each point in the sequence by the same random amount v , uniform on $[0, 1]^d$. Coordinate values greater than one are “wrapped” using modulo-1 division, i.e. if $u_i^{(j)}$ is the j -th coordinate of the i -th element in the original sequence, its randomised counterpart $\hat{u}_i^{(j)}$ is given by

$$\hat{u}_i^{(j)} = (u_i^{(j)} + v^{(j)}) \mod 1 \quad (7.90)$$

where $v^{(j)}$ is the j -th coordinate of v . Thus we can generate arbitrarily many randomised versions of the original sequence by repeated draws of v .

From an implementation perspective, we now have quasi-random Monte Carlo (simulation using the sequence $\hat{u}$) nested inside pseudo-random Monte Carlo (repeated draws of v), and a meaningful standard deviation of the Monte Carlo estimate (which is now an average over the different quasi-random Monte Carlo estimates for each $\hat{u}$) can be obtained at the level of the outer loop.

³⁹ Glasserman (2004) also notes some results in the literature which indicate that randomising quasi-random methods may actually lead to better accuracy.

```

template <class price_process,
          class random_variable,
          class QR_generator> double
RandomShiftQMCMapping<price_process,random_variable,QR_generator>
::mapping(random_variable x)
{
    size_t i;
    double result = 0.0;
    QRgen.reset();
    for (i=0;i<QRgen.number_of_points();i++)
        result += mc_mapping.mapping(QRgen.random(x));
    return result/QRgen.number_of_points();
}

```

Listing 7.26: Mapping a given random shift to a quasi-random Monte Carlo estimate

No new code is required to implement the outer loop — this is simply a generic Monte Carlo algorithm, which can be instantiated by

```

MCGeneric<Array<double,2>,double,
          RandomArray<ranlib::Uniform<double>,double>> (7.91)
          randomQMC(QRfunc,unigen);

```

where `unigen` is a `RandomArray<ranlib::Uniform<double>,double>` with member function `random()` returning an `Array<double,2>` of the appropriate size — as before in `SobolArray`, the convention is to have a $(j \times n)$ `Array` when each draw of the Monte Carlo simulation requires n independent points of dimension j , e.g. for n time steps of a j -dimensional stochastic process. Running

```
randomQMC.simulate(mcgathererQRran,32); (7.92)
```

on the `MCGatherer mcgathererQRran` then results in a reasonably meaningful standard deviation for the Monte Carlo estimator `mcgathererQRran.mean()` being returned by `mcgathererQRran.stddev()`.

The inner, quasi-random simulation loop is contained in the functor

```
boost::function<double (Array<double,2>)> QRfunc (7.93)
```

To instantiate `QRfunc` by

```

QRfunc =
boost::bind(&RandomShiftQMCMapping<GeometricBrownianMotion,
          Array<double,2>,SobolArrayNormal>::mapping,
          &QR_mapping,_1);

```

we need an instance `QR_mapping` of the template class

```

RandomShiftQMCMapping<GeometricBrownianMotion,
          Array<double,2>,SobolArrayNormal> (7.94)

```

The public interface of this class template mirrors `MCMapping` (see Listing 7.5), and it maps a realisation of the random shift v to a quasi-random estimate of a discounted expected payoff. The constructor of `RandomShiftQMCMapping`

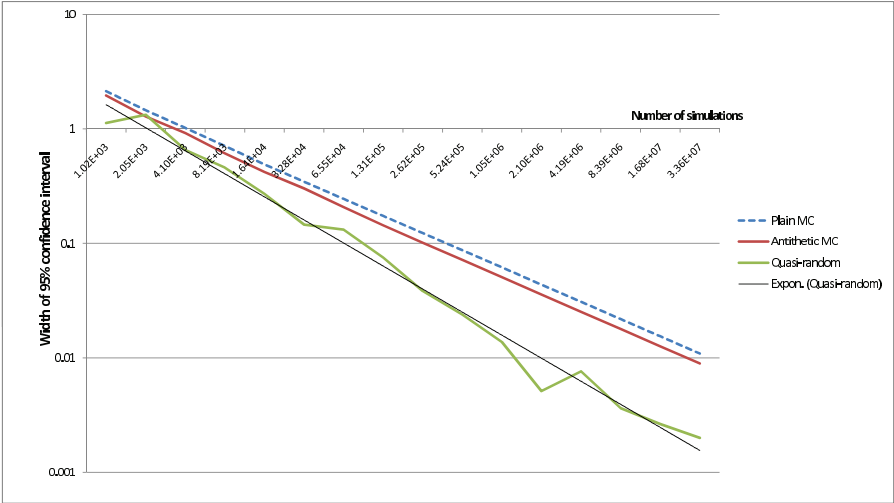


Figure 7.6 *Convergence of confidence intervals for Monte Carlo estimate of Margrabe option price, log/log scale, trendline added for quasi-random case.*

(given in Listing 7.25) creates the appropriate `MCMapping` from objects representing the payoff, the price process of the underlying(s), the term structure of interest rates and the index of the numeraire asset associated with the martingale measure under which the simulation is conducted. The member function `mapping()` (given in Listing 7.26) and analogously `mappingArray()` (for multiple payoffs) calculate the quasi-random estimate for a given shift of the quasi-random sequence generator, e.g. `SobolArrayNormal`, by overloading the member function `random()`, e.g.

```
Array<double,2>& SobolArrayNormal::  
    random(Array<double,2>& shift) (7.95)
```

Listing 7.27 gives an example of pricing a Margrabe option in a two-factor Black/Scholes model using randomised quasi-random Monte Carlo — this is the “randomised” version of Listing 7.24. The full working code is given in `QRMargrabeExample.cpp` on the website. This example program compares pseudo-random Monte Carlo (with and without antithetic sampling) with randomised quasi-random Monte Carlo. Figure 7.6 plots the convergence of the confidence intervals in the three cases — note how quasi-random sampling improves the speed of convergence, while antithetic sampling reduces the level of the initial error, but converges at the same $\mathcal{O}(N^{-\frac{1}{2}})$ rate as “plain” Monte Carlo.

Randomisation allows us to use control variates in combination with quasi-random sampling — this is implemented in the file `QRCVTest.cpp` on the website, for the case of a call option on the arithmetic average. Note that using the template-based implementation of the generic Monte Carlo algo-

```

// Margrabe option price by randomised quasi-random Monte Carlo
// asset price process
GeometricBrownianMotion gbm(underlying);
// create MCPayoffList from MBinaries
boost::shared_ptr<MBinaryPayoff> M1payoff = M1.get_payoff();
boost::shared_ptr<MBinaryPayoff> M2payoff = M2.get_payoff();
MCPayoffList mcpayofflist;
mcpayofflist.push_back(M1payoff);
mcpayofflist.push_back(M2payoff,-K);
// For nested construction of randomised QMC using random shift
MCGatherer<double> mcgathererQRran;
ranlib::Uniform<double> ugen;
/* generator for array of uniform random numbers to shift each coordinate
   of the Sobol points */
RandomArray<ranlib::Uniform<double>,double> unigen(ugen,2,N);
unsigned long n = minpaths;
cout << "randomised QR MC value,95% CI lower bound,";
cout << "95% CI upper bound,Difference in standard errors,CI width\n";
boost::math::normal normal;
double d = boost::math::quantile(normal,0.95);
double CFprice = margrabe.price();
// loop to demonstrate convergence
while (n<=maxpaths) {
    // Nested construction of randomised QMC using random shift
    SobolArrayNormal sobolr(2,N,n/32);
    /* MCMMapping to map uniform random numbers to a shifted quasi-random
       simulation estimate of discounted payoffs */
    RandomShiftQCMMapping<GeometricBrownianMotion,Array<double,2>,
                          SobolArrayNormal>
        QR_mapping(sobolr,mcpayofflist,gbm,ts,numeraire_index);
    // mapping functor
    boost::function<double (Array<double,2>)> QRfunc =
        boost::bind(&RandomShiftQCMMapping<GeometricBrownianMotion,
                                             Array<double,2>,
                                             SobolArrayNormal>::mapping,
                                             &QR_mapping,_1);
    // generic Monte Carlo algorithm object
    MCGeneric<Array<double,2>,double,
              RandomArray<ranlib::Uniform<double>,double> >
        randomQMC(QRfunc,unigen);
    mcgathererQRran.reset();
    randomQMC.simulate(mcgathererQRran,32);
    cout << mcgathererQRran.mean() << ','
        << mcgathererQRran.mean()-d*mcgathererQRran.stddev() << ','
        << mcgathererQRran.mean()+d*mcgathererQRran.stddev() << ','
        << (mcgathererQRran.mean()-CFprice)/mcgathererQRran.stddev() << ','
        << 2.0*d*mcgathererQRran.stddev() << endl;
    n *= 2; }

```

Listing 7.27: Pricing a Margrabe option by randomised quasi-random Monte Carlo simulation